



GIT AND GITHUB DESKTOP



Presentación de apoyo (lo importante es poner el práctica lo que esta en el repositorio)



QUE ES EL CONTROL DE VERSIONES

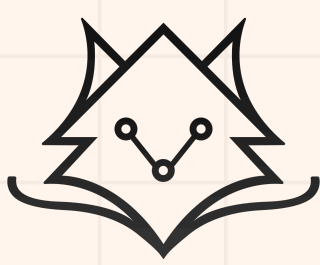
Es un sistema que registra y **gestiona los cambios** realizados en un **archivo** o conjunto de archivos a lo **largo del tiempo**.

El control de versiones existe **para resolver el caos** que ocurre al trabajar en proyectos complejos o en equipo. Sin esta herramienta, los proyectos sufren de problemas graves.

GIT VS GITHUB

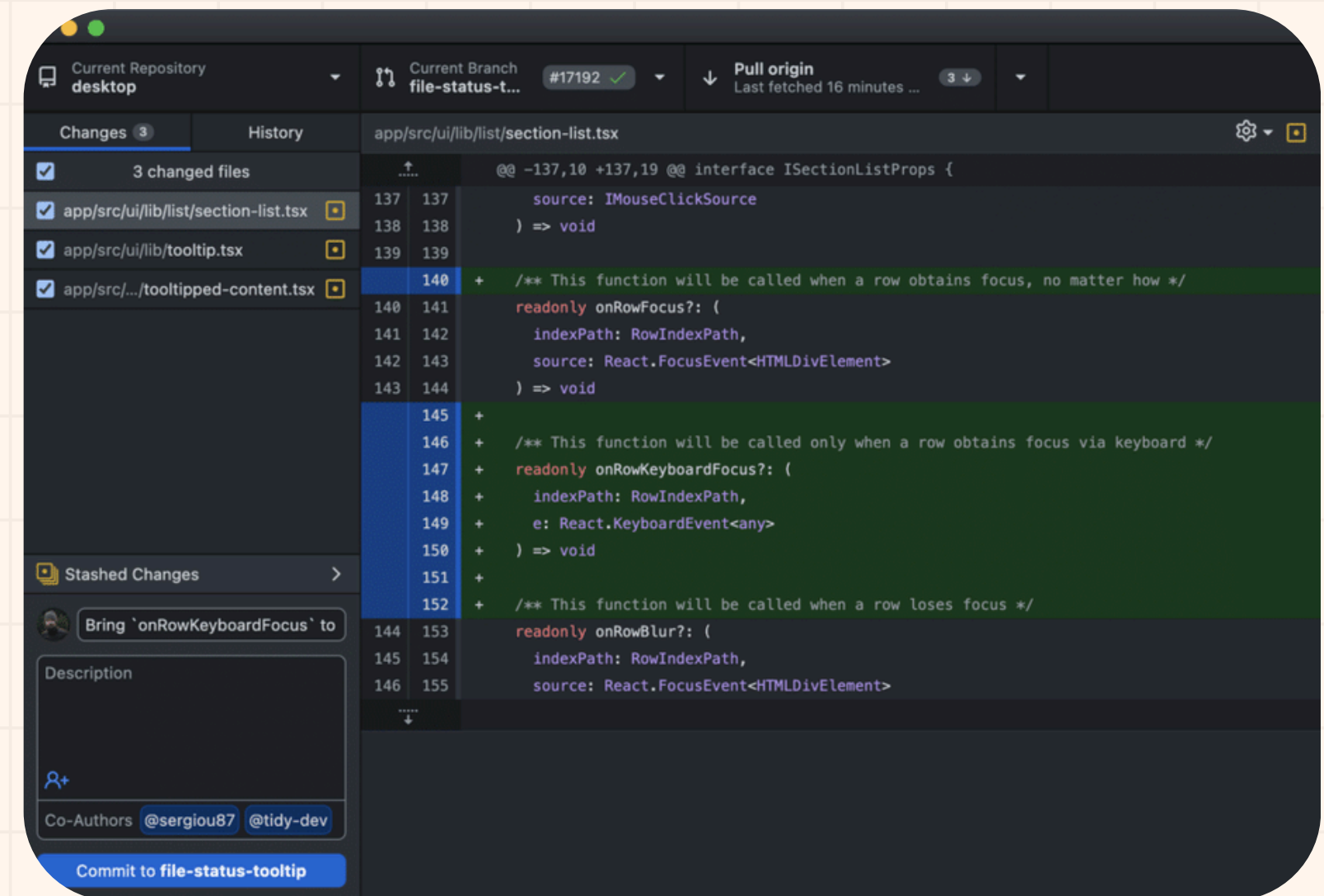
- **Git (La Herramienta):** Es un programa que instalas en tu computadora. Se encarga de **guardar el historial** de todos los cambios que haces en tus archivos de código. Funciona de **manera local** y **no necesita internet**.
- **GitHub (La Plataforma):** Es una página **web en la nube** que **utiliza Git**. Es el lugar donde subes ese historial para **tener un respaldo** en internet, compartir tu código y poder trabajar en equipo con otras personas al mismo tiempo.

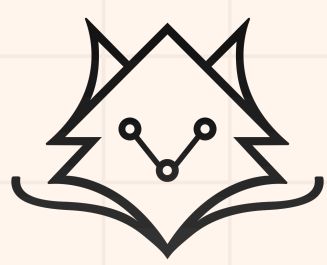




PORQUE GITHUB DESKTOP

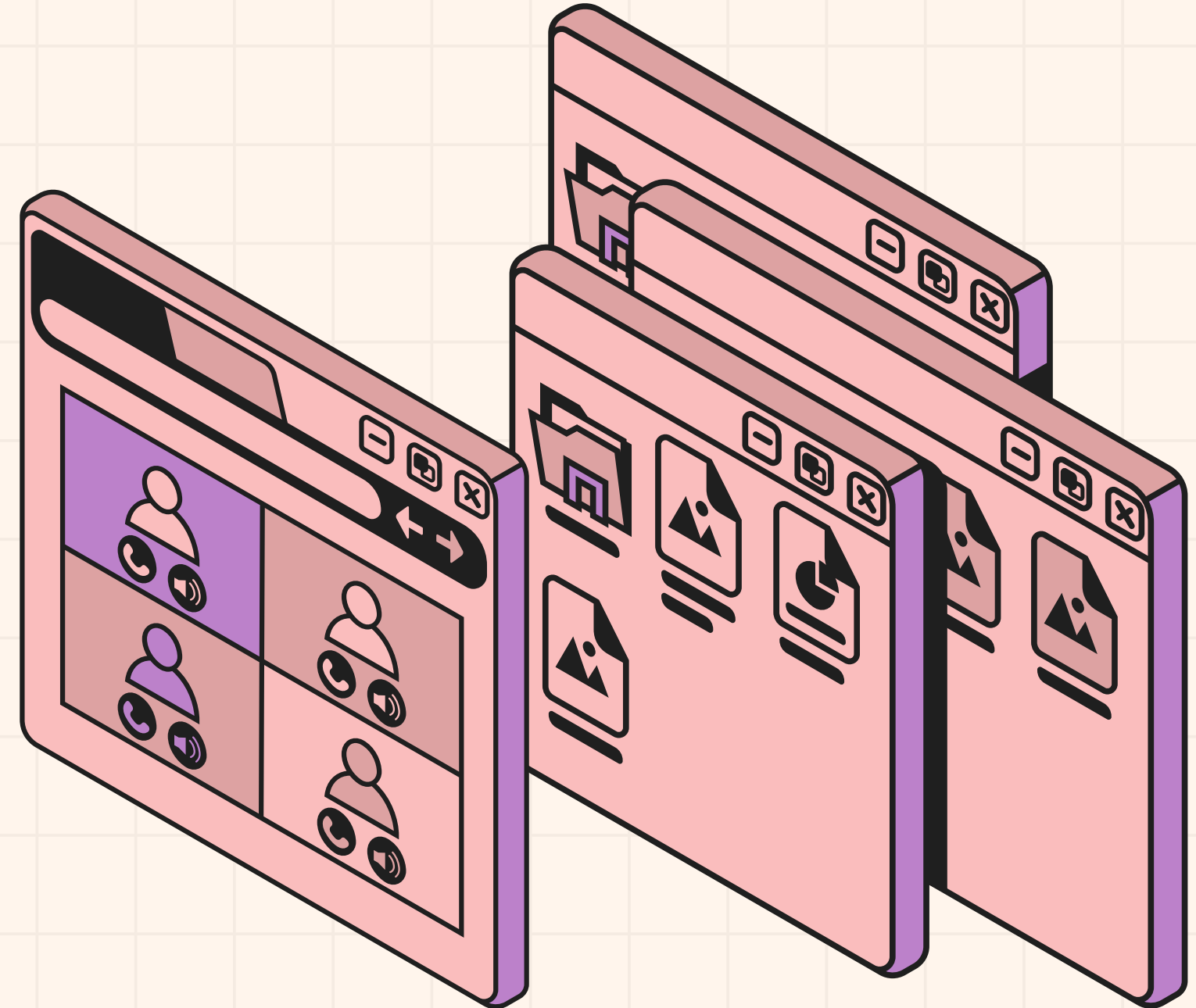
- **Curva de aprendizaje casi nula:** Su interfaz es **extremadamente minimalista**. Es la **herramienta ideal para principiantes**, que necesitan subir archivos al repositorio sin aprender comandos técnicos.
- **Es 100% gratuito**
- **Integración nativa:** Como es de la misma casa, funciona de maravilla con tu cuenta de GitHub. Puedes crear un Pull Request o revisar las alertas de seguridad de tu código directamente desde la aplicación sin abrir el navegador.





REPOSITARIOS QUE SON

Un repositorio **es la carpeta principal** y oficial de **tu proyecto**. No solo guarda todos tus archivos, sino que **funciona** como una "**máquina del tiempo**" que **almacena el historial** completo de cada cambio que se les ha hecho. Este puede **vivir de forma local** en tu computadora o estar **respaldado en la nube** (como en GitHub) para que puedas trabajar en equipo.



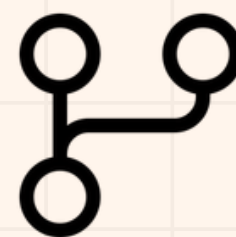


PROPIEDADES



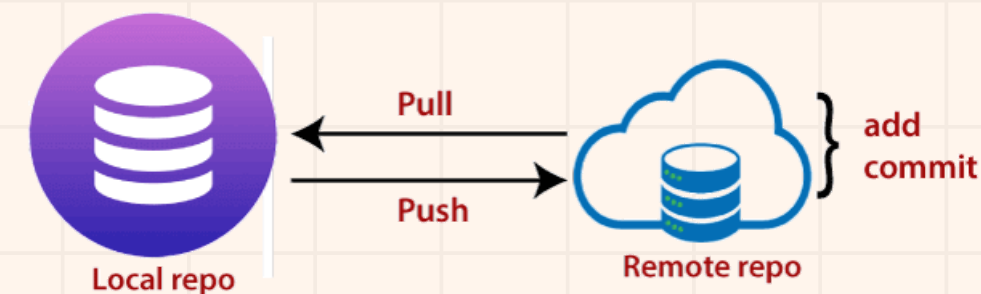
COMMIT

Es un **objeto de datos inmutable** que almacena una instantánea exacta (snapshot) del estado de tus archivos en un momento específico. Se identifica mediante un hash criptográfico.



BRANCH

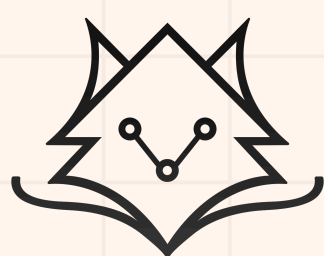
Es un entorno de **desarrollo aislado** que bifurca el historial principal para que puedas **crear** o **modificar código** sin afectar el trabajo de los demás



PUSH AND PULL

Push: transmite tus commits y objetos locales **hacia un repositorio remoto**, actualizando los punteros de sus ramas.

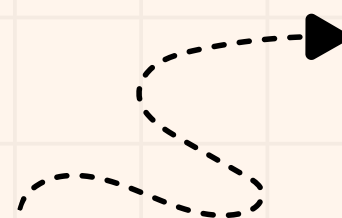
Pull: es una operación combinada que **descarga los cambios** del servidor remoto y **los integra inmediatamente** en tu rama local activa



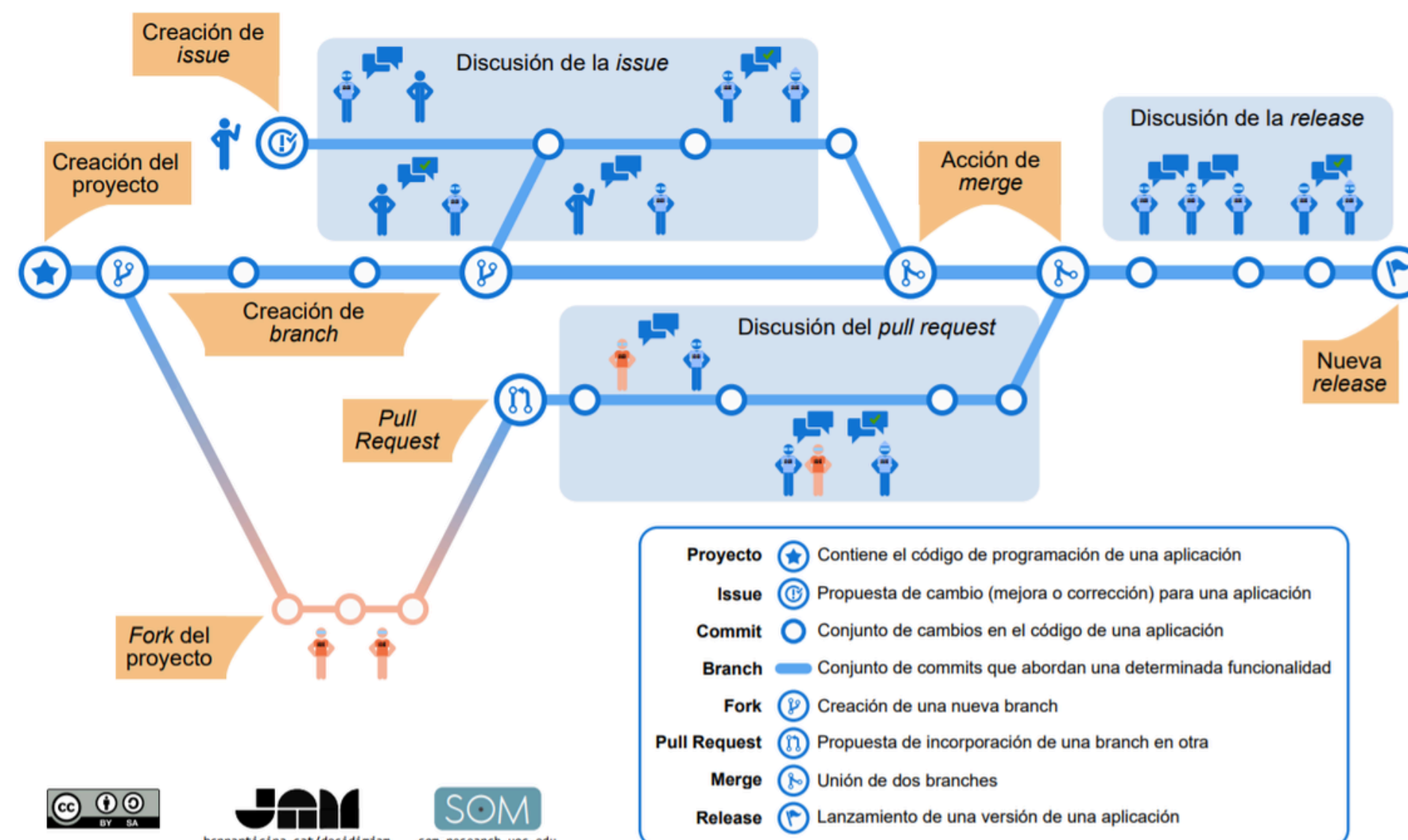
LA BUROCRACIA DE GIT

La burocracia se basa en **Pull Request (PR)**, la cual es una **solicitud formal** en plataformas como GitHub/Gitkraken para **proponer que los cambios** que hiciste en tu rama de trabajo se integren a la rama principal del proyecto (como main).

Sirve para abrir un **espacio de revisión** donde tu equipo **puede examinar tu código**, hacer comentarios, **sugerir mejoras** y **aprobar los cambios antes de fusionarlos** definitivamente con el código oficial.



¿Cómo se desarrolla en GitHub?
¿Cuáles son los principales elementos de discusión?



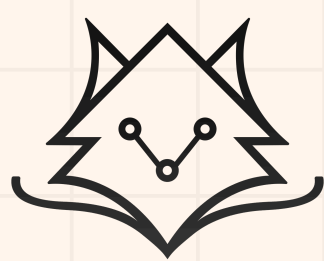


GIT AND TERMINAL



Slides del curso avanzado de Git

Presentación de apoyo (lo importante es poner el práctica lo que esta en el repositorio)



REGLAS

“LA TERMINAL NO ES EL TEMA,
ES EL VOLANTE”



- Cero código
- Cero botones de Desktop
- Todo escrito en Markdown

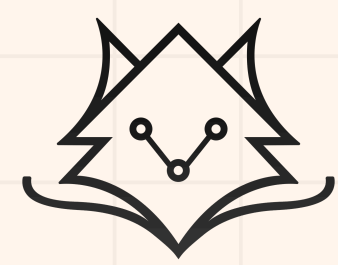
Herramientas:

Terminal → **git and gh**
Github.com → **Para configurar y revisar**
VS Code → **graph and gitlens**



Msj del fantasma de Misa → Misa odia
Github Desktop...todo el terminal guys :)





MAPA DE LA SESIÓN

4 horas · **240 min** · sin margen

1ª MITAD

1

20'

El botón era un comando

2

30'

Local y origen

3

15'

Tu primer issue

4

35'

El ritual diario

PAUSA

Descanso · 10'

2ª MITAD

5

30'

Las reglas de la casa

6

35'

Historia limpia

7

35'

Conflictos en equipo

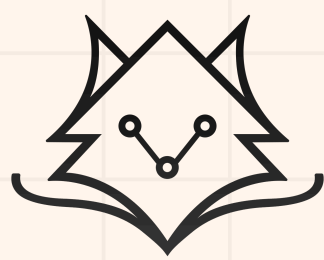
8

15'

Ciclo completo sin red

CIERRE

+ tarea · 15'



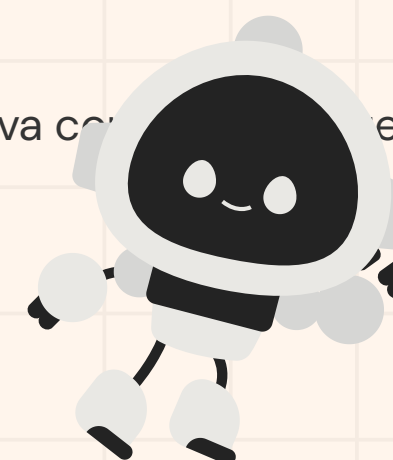
CHECK IN

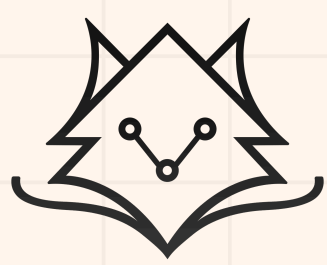
Antes de empezar, **todas** las compus:

```
Terminal
$ gh auth status
github.com
✓ Logged in to github.com account ana-gh
- Active account: true
- Git operations protocol: https
```

***“¿Te salió la palomita verde? Sigue.
¿No? Levanta la mano: te ayuda un
asistente.”***

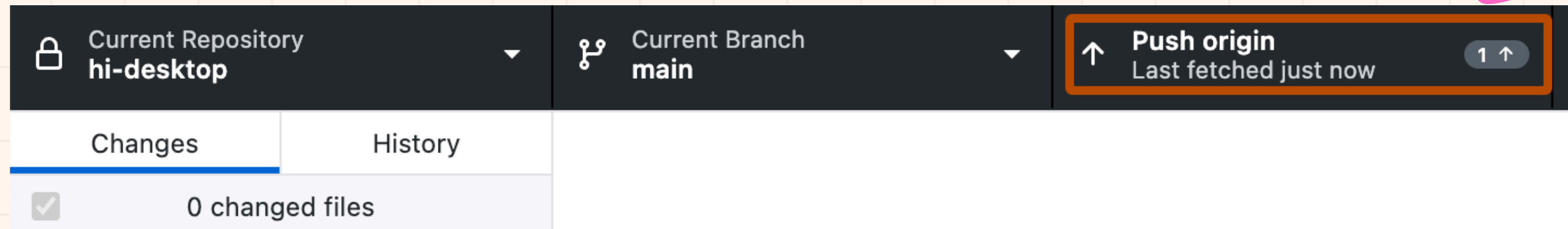
No se pausa al grupo: quien no pase se va con el asistente

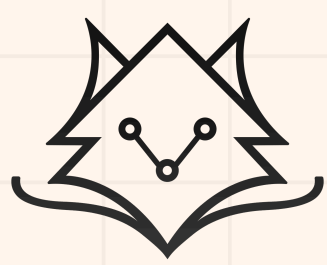




EL BOTÓN ERA EL COMANDO

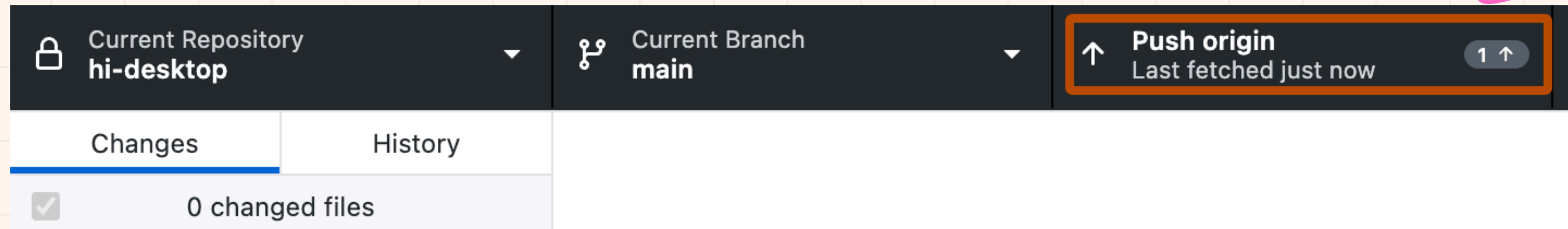
Qué hace este
botón cuando lo
presionas?



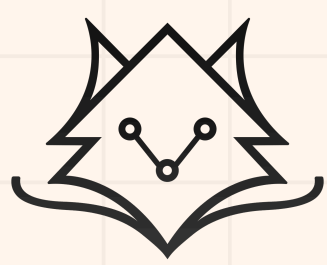


EL BOTÓN ERA EL COMANDO

Qué hace este
botón cuando lo
presionas?



git push origin



PRIMEROS COMANDOS

git --version: Muestra en la pantalla el número de versión de Git que tienes instalado en tu computador

gh auth status: Te verifica y muestra el estado actual de tu autenticación para todos los servidores y cuentas configuradas

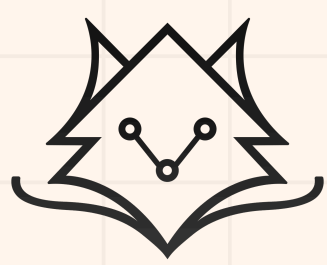
git config user.name: Comando de Git que sirve para definir o consultar el nombre de autor que aparecerá registrado en cada uno de tus cambios o commit.

“La terminal no es difícil, es
honesta”



Mientras más usen los comando, más rápido se les va a quedar

IMPORTANT



EL CICLO DEL COMMIT

git init (New Repository) → **git status**
(Panel Changes) → **git add** (Marcar Casilla)
→ **git commit -m** (Summary + Commit) →
git log --oneline (History) → **git diff**



```
$ git status
Untracked files:  notas.md
$ git add notas.md
$ git status
Changes to be committed:  new file: notas.md
```

rojo = aún no lo marcas · verde = listo para el commit

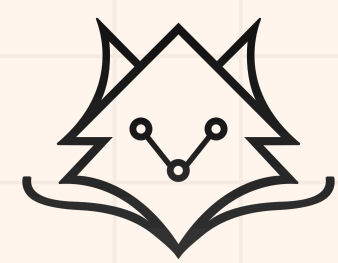
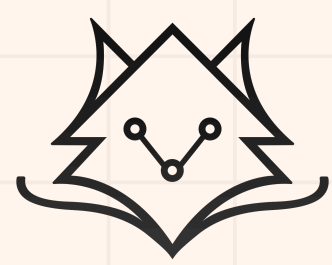


TABLA DE TRADUCCIÓN

SLIDE FIJA se queda proyectada

En GitHub Desktop	En la terminal
File → New Repository	<code>git init</code>
Clone repository	<code>gh repo clone usuario/repo</code>
Panel Changes	<code>git status</code> · <code>git diff</code>
Marcar la casilla	<code>git add archivo</code>
Summary + Commit	<code>git commit -m "mensaje"</code>
Pestaña History	<code>git log --oneline --graph</code>
New Branch	<code>git switch -c rama</code>

En GitHub Desktop	En la terminal
Cambiar de rama	<code>git switch rama</code>
Merge into Current Branch	<code>git merge rama</code>
Publish repository	<code>git remote add origin ...</code> <code>git push -u origin main</code>
Push origin	<code>git push</code>
Fetch origin	<code>git fetch</code>
Pull origin	<code>git pull</code>
Create Pull Request	<code>gh pr create</code>



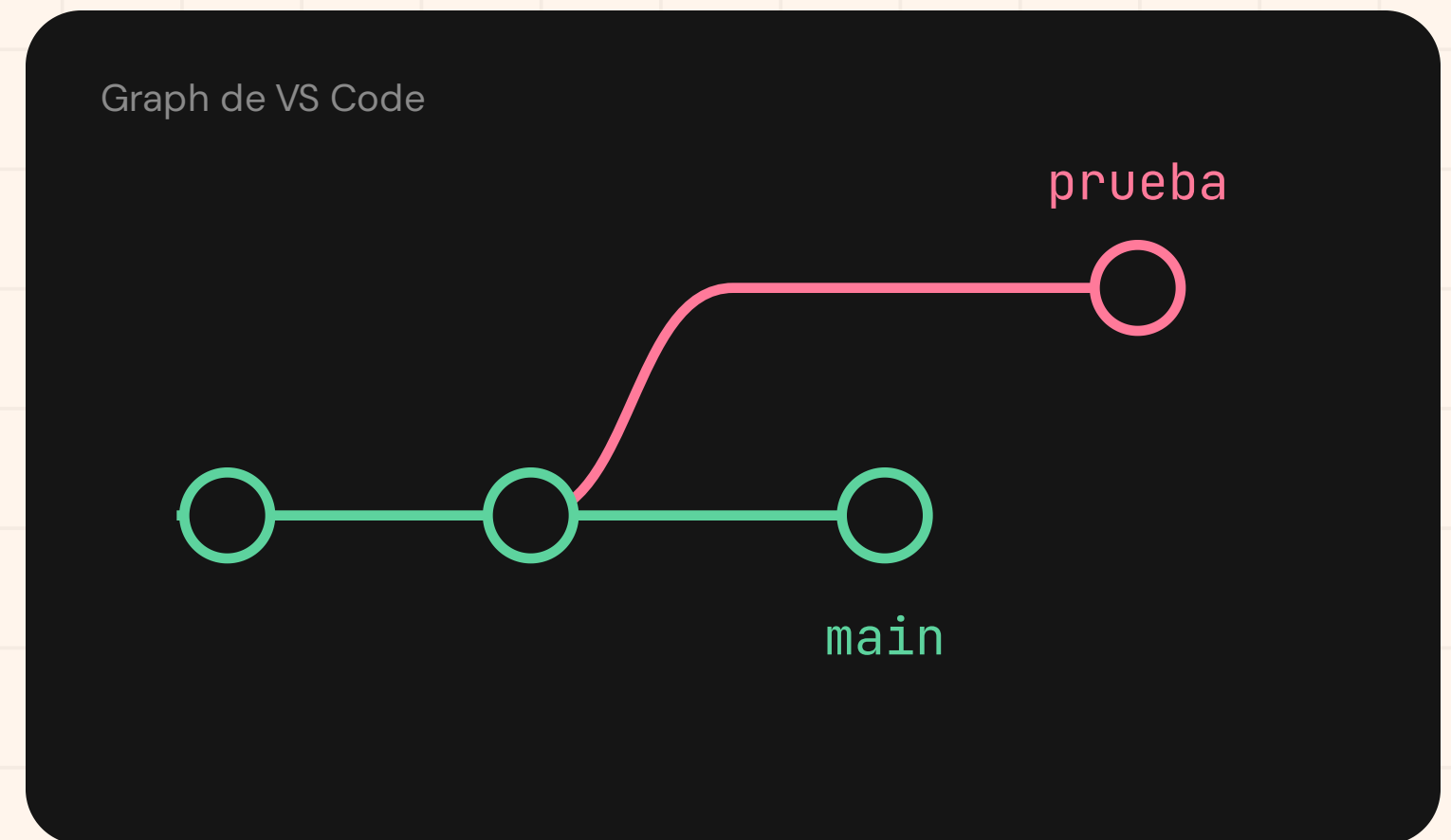
REPASO DE RAMAS

`git switch -c prueba` → “abro un desvío”

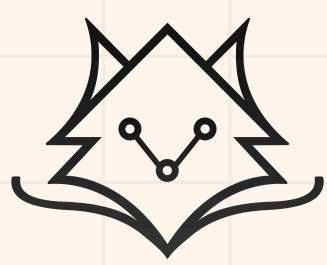
(cambio + commit)

`git switch main` → “el cambio desaparece”

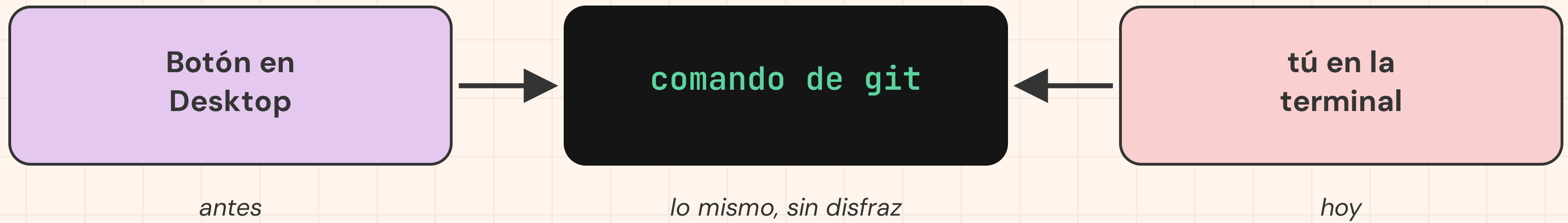
`git switch prueba` → “el cambio reaparece”



“Este repo es solo suyo: es su cuaderno de práctica.”



EL BOTÓN ERA UN COMANDO



→ Hay dos copias del repo. ¿Cuál es la de verdad?

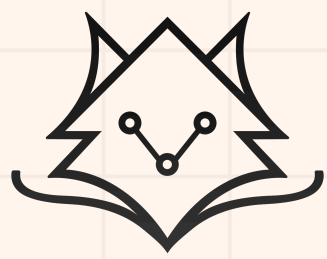


CUANDO EL REPO SE **SUBE A GITHUB** ¿CUÁL ES LA COPIA DE VERDAD?



Slides del curso avanzado de Git

Presentación de apoyo (lo importante es poner el práctica lo que esta en el repositorio)



EL LÍDER PREPARA LA BASE

README.md:

Una serie de especificaciones o guía para el equipo:

Título

Tabla de integrantes (Solo encabezado: Nombre, Carrera, User de Github)

Dos plantillas:

.github/ISSUE_TEMPLATE/tarea.md

.github/pull_request_template.md



Crea un repo en GitHub (vacío)

Tres comandos de publicar:

git remote add origin... → “Le presento a mi repo su dirección en internet”

git branch -M main → “Le pongo el nombre que Github espera”

git push -u origin main → “Subo main y la amarro con la de origin”

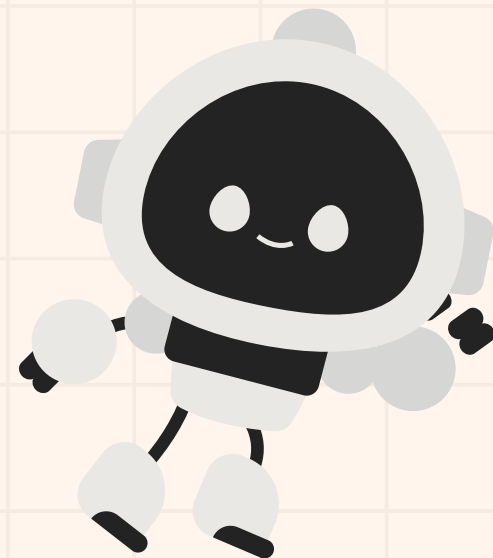
“Subió la historia completa. Y no subió prueba: solo sube lo que tú le dices”

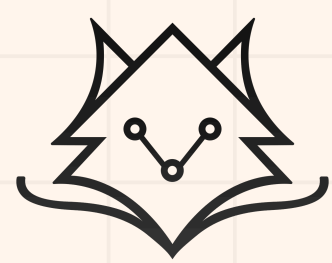
Que es cd?. *Hablar de los comandos básicos de la terminal*

Líder → agrega **Collaborators**

Equipo → **cd .. + gh repo clone líder/equipo-[nombre]**

“Clonar es el único momento en que no hacen git init”





¿DÓNDE ESTOY?

`pwd` → “¿en qué carpeta estoy?”

`ls` → “¿qué hay aquí?”

`cd carpeta` → “entro a la carpeta”

`cd ..` → “subo un nivel”

Tab autocompleta · **↑** repite el último comando

Escritorio/

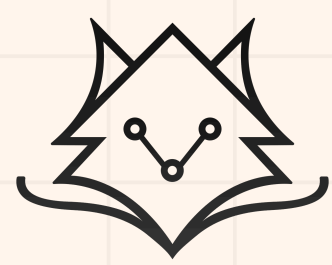
├── practica-terminal/

└── equipo-[nombre]/

`cd ..`

(ahí es donde clonan el repo del equipo)

“La terminal siempre está parada en una carpeta.”



CREAR EL REPO VACÍO

“Tiene que nacer vacío: la historia ya existe en su compu.”

¿Alguien marcó README? Settings → Danger Zone → borrar y crearlo de nuevo

NEW REPOSITORY

Repository name

☒ **Public** ☐ Private

☐ Add a README file

NO

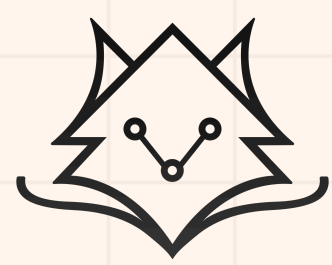
☐ Add .gitignore

NO

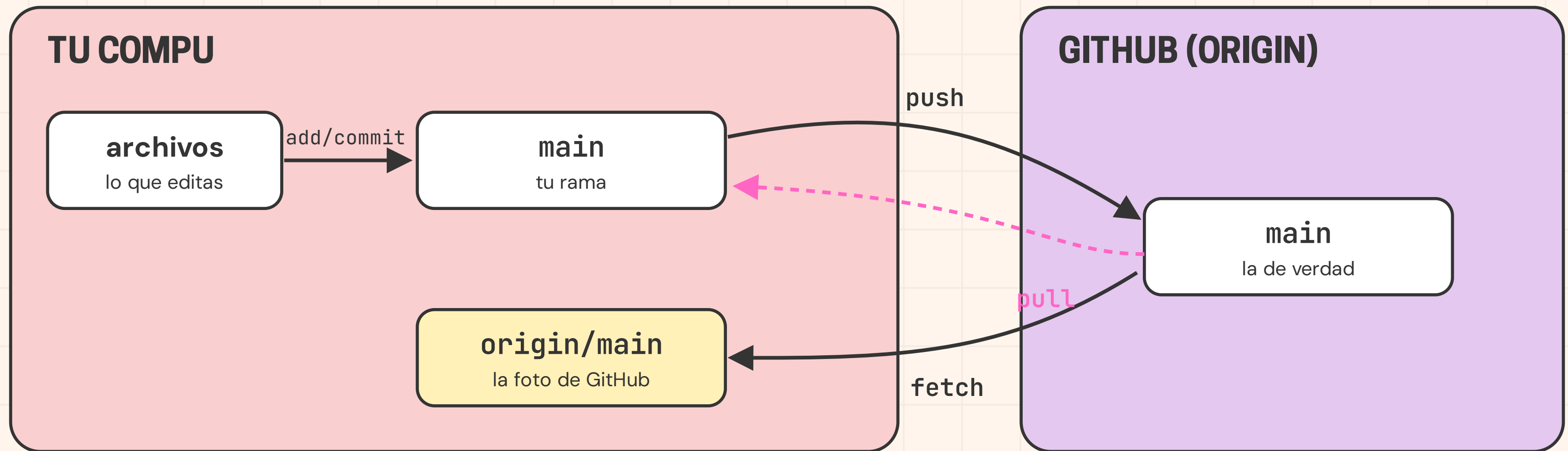
☐ Choose a license

NO

Create repository

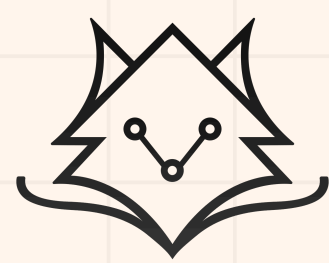


MAIN VS ORIGIN/MAIN



main = tu rama · **origin/main** = la foto que tu compu tiene de GitHub

`git remote -v` → en el repo del equipo: **origin** ✓ · en practica-terminal: **nada**



LA FOTO VIEJA

¿Es verdad? →

“git status no le pregunta a GitHub, le pregunta a tu foto de GitHub.”

`fetch` → actualiza la foto

`pull` → actualiza la foto **y** trae los cambios

```
Terminal · repo del equipo

$ git status
Your branch is up to date with 'origin/main'.

$ git fetch
$ git status
Your branch is behind 'origin/main' by 1 commit.

$ git pull
Fast-forward
```

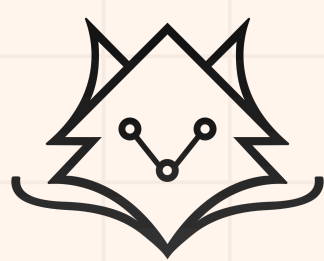
→ ¿Quién decide qué hay que hacer y quién lo hace?



“Nadie cocina sin comanda. Nadie trabaja sin issue.”

título · descripción ·
responsable · #N

[illegible]



CREAR ISSUE

Pasos en Github:

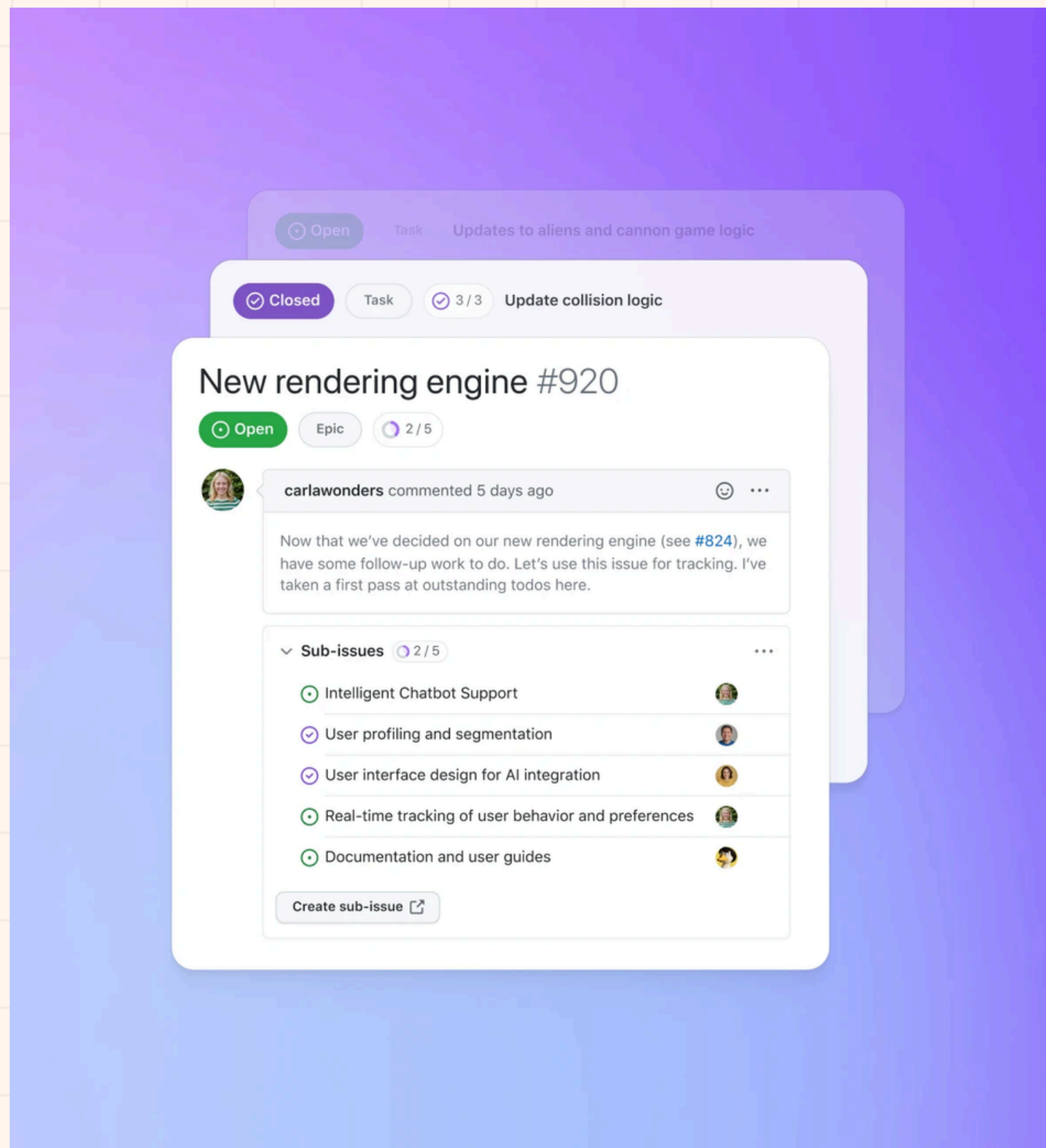
Issues → New issue → plantilla Tarea → título 'Agregar perfil de [nombre]' → 2 preguntas → Assign yourself → Create

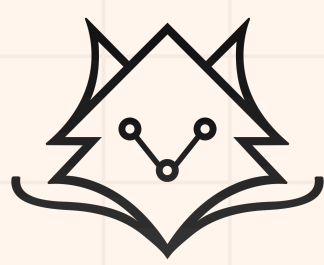
Pasos en Terminal:

gh issue list (o gh issue create como alternativa).
"Anoten su número"

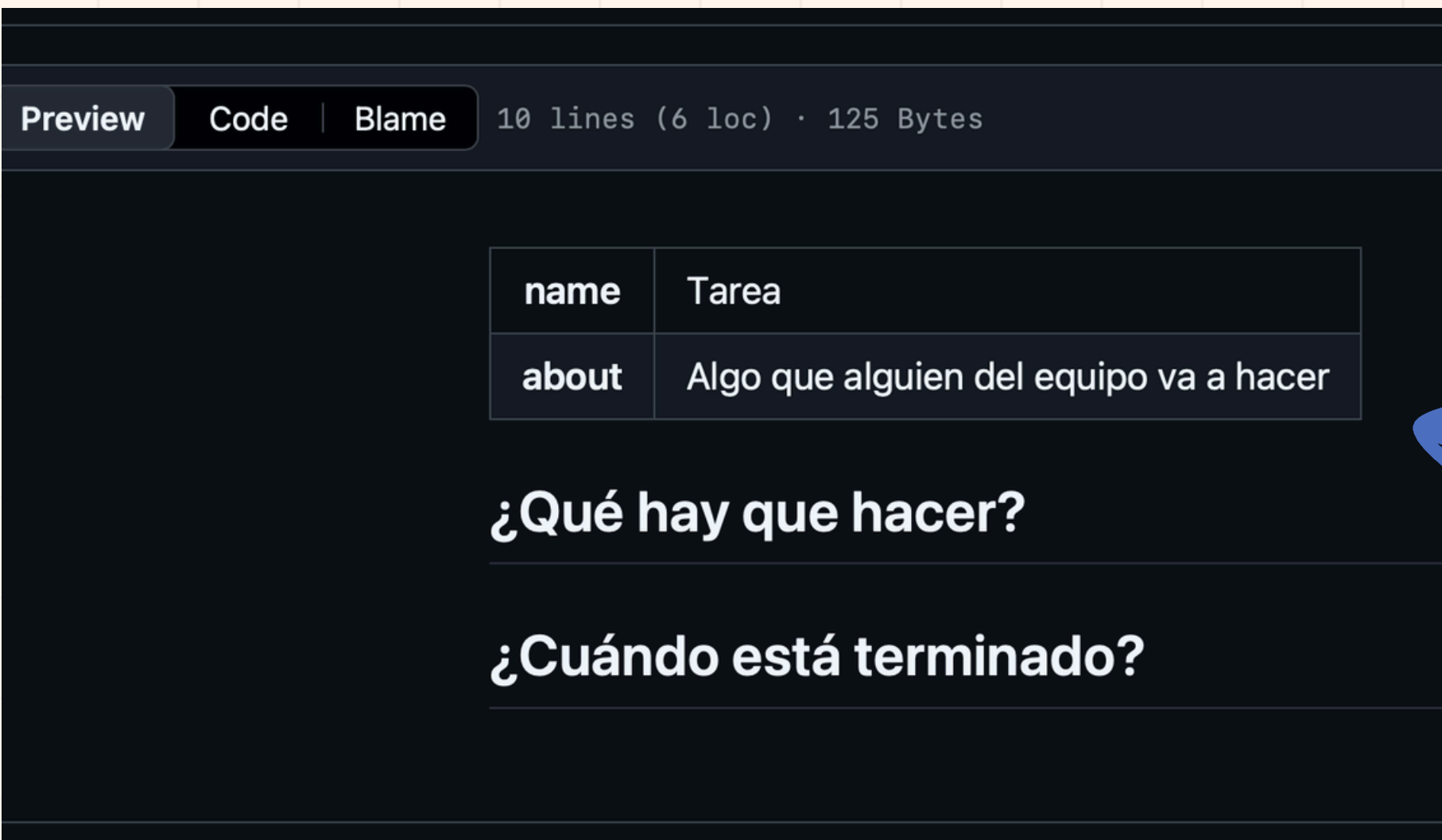


```
$ gh issue list
#3   Agregar perfil de Ana
#4   Agregar perfil de Luis
```





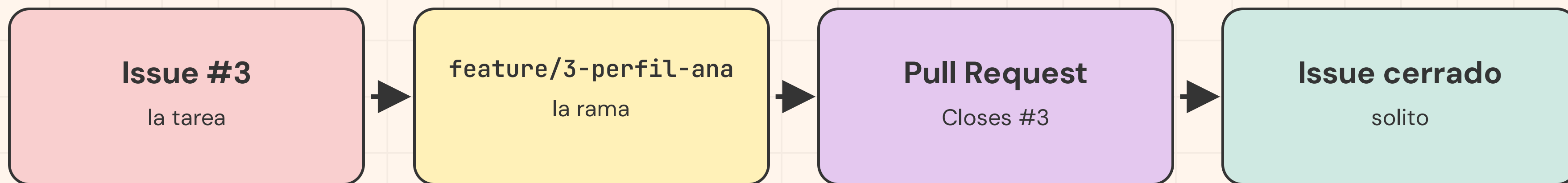
LA CONFIGURACIÓN ES UN ARCHIVO



*Lo que está en `.github/`
configura GitHub y viaja
con el repo*

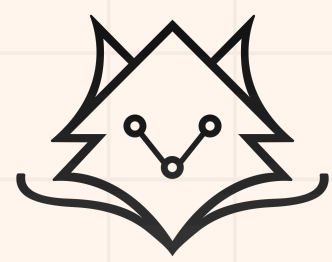


DEL ISSUE AL CIERRE



El número del issue viaja en el nombre de la rama y en el PR.

→ El ritual que van a repetir **cada vez** que se sienten a trabajar.



EL RITUAL EN TRES MOMENTOS

SLIDE FIJA

1 ANTES

Ponerme al día

```
git switch main  
git pull  
git switch -c ...
```

2 DURANTE

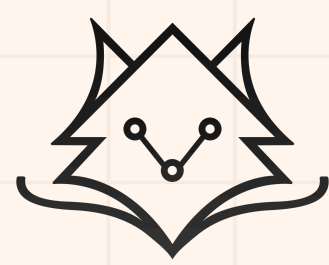
Commits pequeños y
mirar dónde estoy

```
git status  
git add  
git commit -m  
git log --oneline --graph
```

3 DESPUÉS

Subir, pedir revisión,
integrar, limpiar

```
git push  
gh pr create  
revisión + merge  
limpiar ramas
```



EL RITUAL DIARIO

Antes de empezar:

`git switch main` → “me paro en la base”

`git pull` → “me pongo al día”

`git switch -c feature/3-perfil-ana` → “abro mi desvío desde un main fresco”

Anatomía del Branch Name:

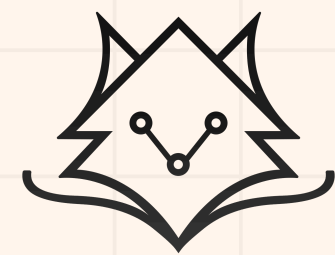
feature / número del issue / descripción

Durante:

`git status + git add + git commit -m “docs:...” + git diff + git log --oneline --graph`

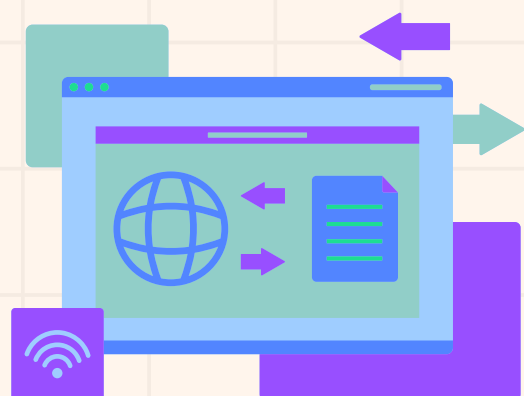
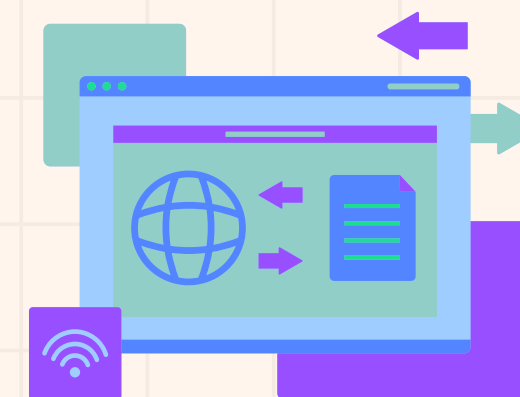


“Main es la mesa donde se sirve, no la tabla donde se corta”



PREFIJOS

docs:	documentos
fix:	correcciones
feat:	algo nuevo



***“Commits pequeños y seguidos
son tus puntos de guardado”***



SUBIR Y ABRIR PULL REQUEST

`git push -u origin feature/3-perfil-ana`



Que hace el `-u`?

`gh pr create --web` → captura el formulario con la plantilla

*El título del PR usa la convención de commits (será historia de ***main***) y ***Closes #3*** cierra el issue

OPEN A PULL REQUEST · PLANTILLA

Título

docs: agrego perfil de Ana

¿Qué cambia?

Agrego mi perfil en integrantes/

Closes #3 ← **cierra el issue**

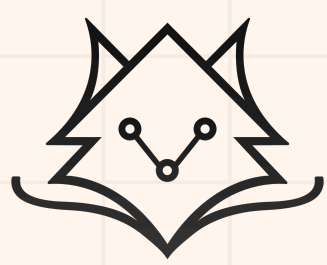
Create pull request

Alternativa en una línea:

```
$ gh pr create --base main \
  --title "docs: agrego perfil de Ana" \
  --body "Closes #3"
```

El título del PR será el commit en main: que se lea solo

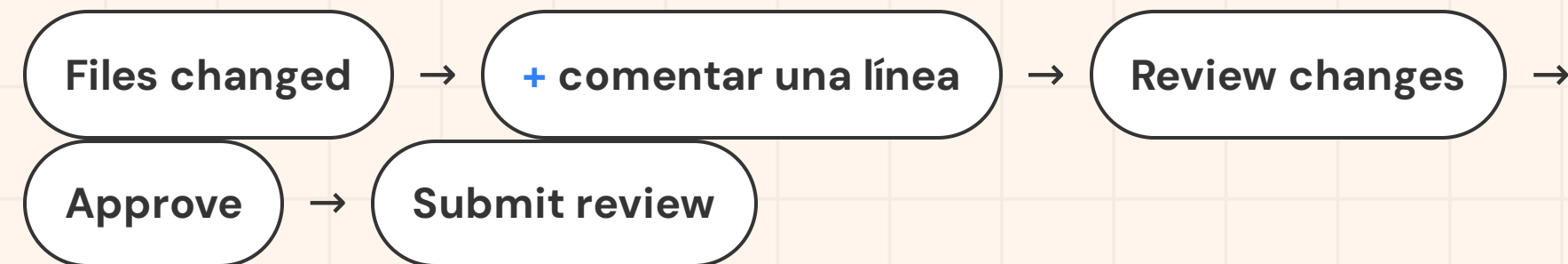
`-u` = amarro mi rama con la de origin → después basta **`git push`**



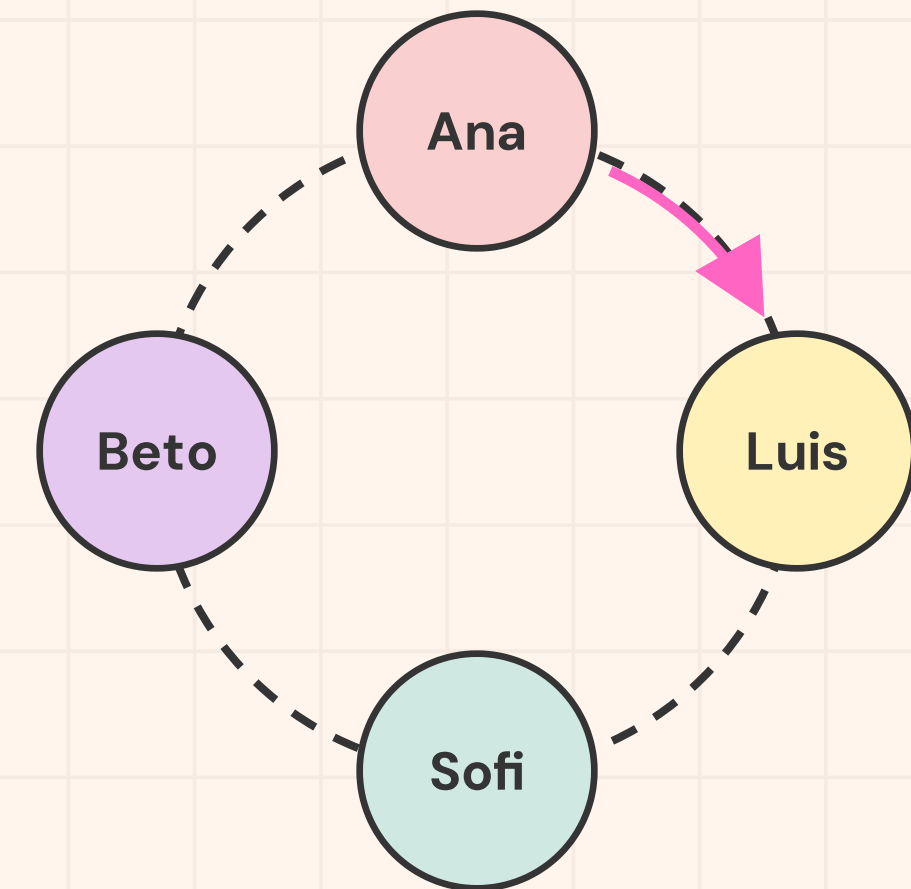
REVISAR EL PR DE UN COMPAÑERO

ORIGIN · GITHUB

En GitHub.com:



“El mismo verde y rojo de siempre, ahora en la web.”



Cada quien revisa al **de su derecha**



INTEGRAR Y LIMPIAR

 TU COMPU

`git switch main` → “me paro en la base”

`git pull` → “traigo lo integrado”

`git branch -d feature/3-perfil-ana` → “borro mi desvío”

`git fetch --prune` → “borro la foto de la rama que ya no existe”

 ORIGIN · GITHUB

Merge pull request



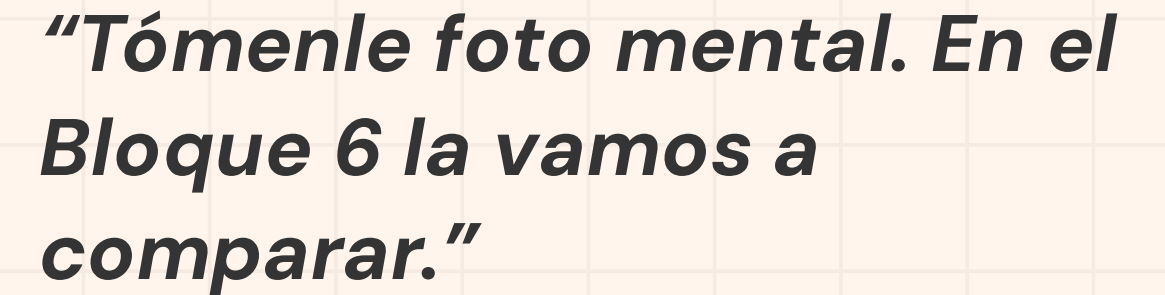
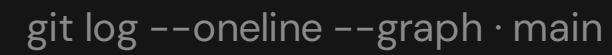
Confirm merge

Delete branch



El issue **#3** se cerró solo

*“Una rama es un desvío
temporal.”*



Page 34



GITLENS: ¿QUIÉN ESCRIBIÓ ESTO?

1 · CURRENT LINE BLAME

```
| Ana | Sistemas |  
Luis, hace 2 h · docs: base del  
equipo
```

2 · HOVER

```
Luis · hace 2 horas  
docs: base del equipo  
a1b2c3d
```

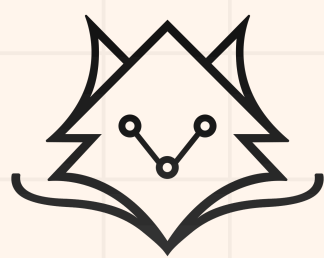
3 · CODELENS

```
Luis, hace 2 h | 3 autores  
# Equipo Lobo  
## Integrantes
```

Hoy NO se usan: Explain · Connect to GitHub · Open in Commit Graph · Revert · Reset ·
Rebase

“GitLens solo mira.”

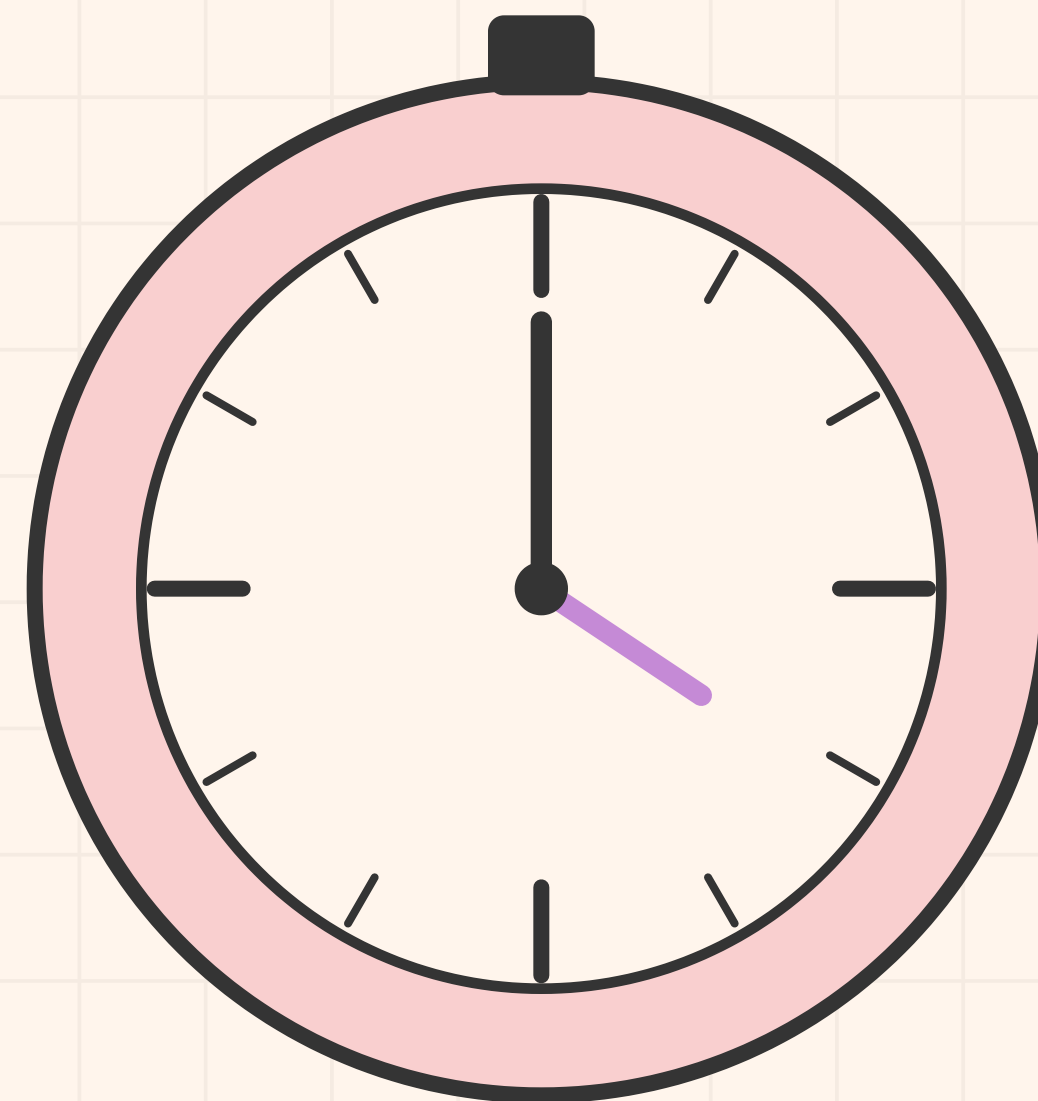
→ ¿Qué impide el atajo y qué mantiene la historia legible?

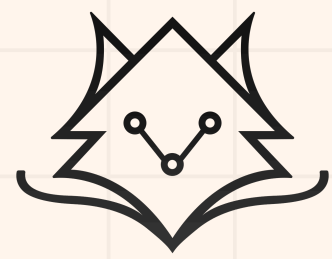


DESCANSO

10 MINUTOS

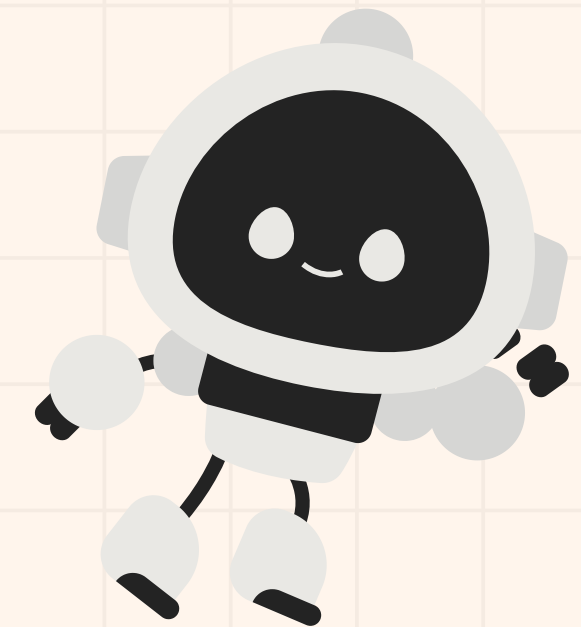
Regreso: _____

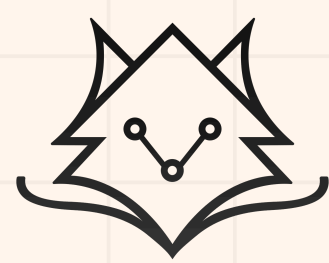




¿QUÉ IMPIDE EL ATAJO?

¿QUÉ MANTIENE LA HISTORIA LEGIBLE?



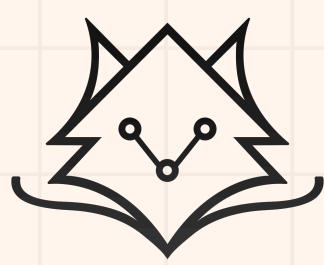


LA PUERTA ABIERTA

“Nadie revisó esto. Nadie sabe por qué se hizo.”

```
Terminal · el líder, en main
$ git commit -am "docs: cambio directo"
[main 5e6f7a8] docs: cambio directo
$ git push
To github.com:lider/equipo-lobo.git
   3c4d5e6..5e6f7a8  main → main
```

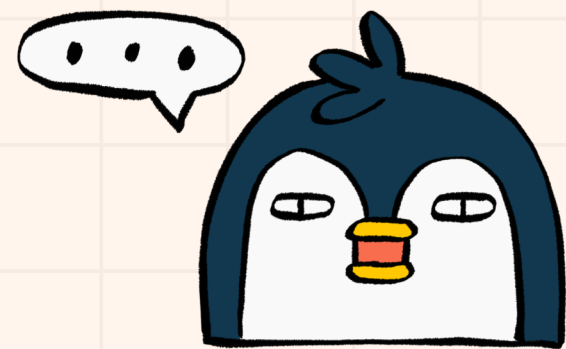
Funcionó... y ese es el problema.



PROPIEDADES PULL REQUEST

MERGE COMMIT

Toma todos los commits de tu rama de trabajo y los une a la rama principal creando un nuevo commit especial de "fusión" que enlaza ambos historiales. Este método se utiliza cuando deseas conservar la historia exacta.



SQUASH AND MERGE

Toma todo tu trabajo y comprime todos los commits de tu rama secundaria en un único y nuevo commit que se añade a la rama principal.

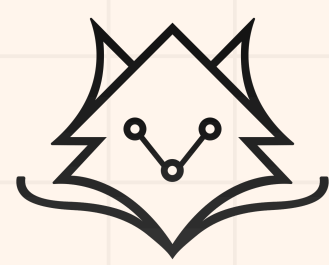
LA QUE USAREMOS



REBASE AND MERGE

Aplica tus commits uno por uno al final de la rama principal sin crear un commit extra de fusión. Es ideal para mantener un historial perfectamente lineal y visualmente limpio, conservando el detalle original de cada paso.





CÓMO ENTRA EL TRABAJO

Settings → General → Pull Requests

☐ Allow merge commits

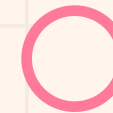
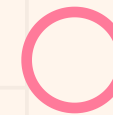
☒ **Allow squash merging**

Default message → **Pull request title and description**

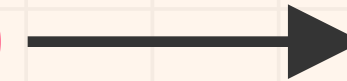
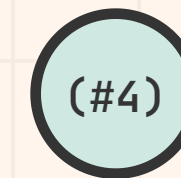
☐ Allow rebase merging

☒ **Automatically delete head branches**

tu rama

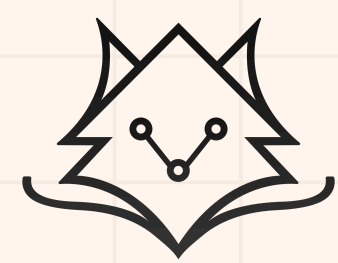


main



SQUASH = APLASTAR

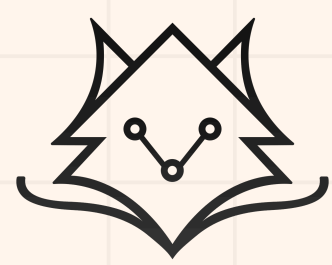
Toda la rama entra a main como **un solo commit**, con el título del PR y su **(#número)**.



EL RULESET PROTEGER-MAIN

Settings → Rules → Rulesets → New branch ruleset · **Target: default branch**

Regla (lo que marcas)	En lenguaje de equipo
Restrict deletions	“nadie borra main”
Block force pushes	“nadie reescribe main”
Require linear history	“main es una línea recta, sin nudos”
Require a pull request · 1 aprobación	“todo entra por PR con un visto bueno”



NI EL DUEÑO

“La regla no es para desconfiar de alguien, es para que nadie tenga que acordarse.”



Terminal · el líder, otra vez en main

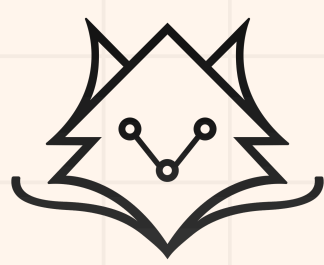
```
$ git push
remote: error: GH013: Repository rule violations found
remote: - Changes must be made through a pull request.
! [remote rejected] main → main (push declined)
```

PARA LIMPIAR SU MAIN LOCAL

```
git reset --hard origin/main
```

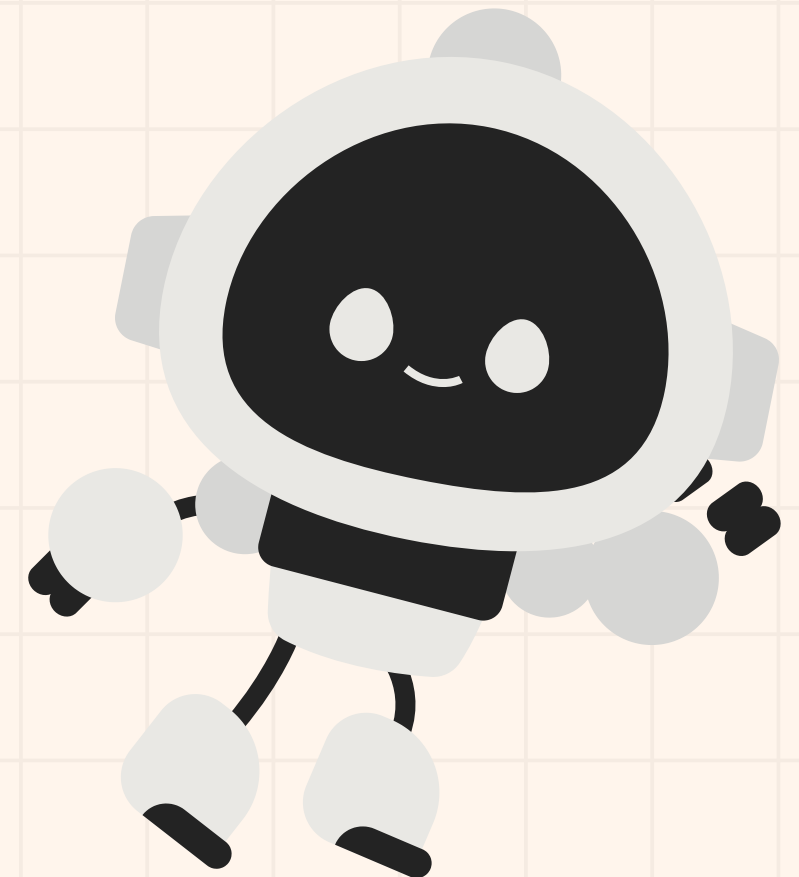
→ “mi main = la foto de origin”

⚠ **Tira lo local que no esté en origin.**



CONTRATAR AL ROBOT

“Una receta: en cada PR, GitHub presta una compu, sigue la receta y reporta.”



Actions



set up a workflow yourself



revision.yml

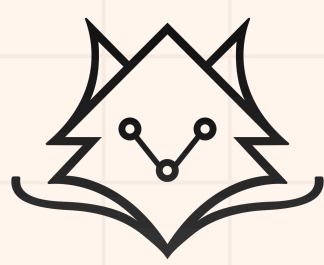


pegar la receta



Sin marcadores de conflicto — Successful

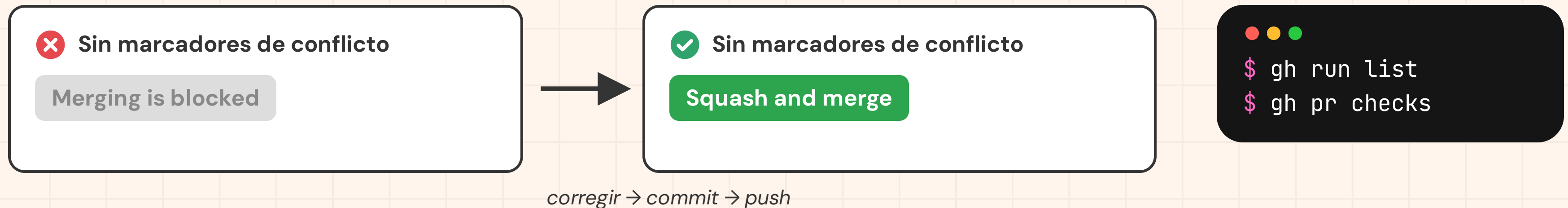
Hasta el robot entró por PR (y el botón ya dice
Squash and merge)

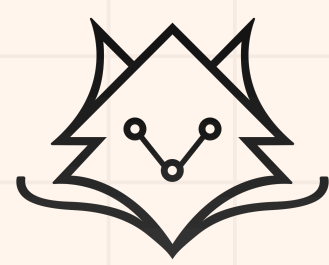


HACER OBLIGATORIO AL ROBOT

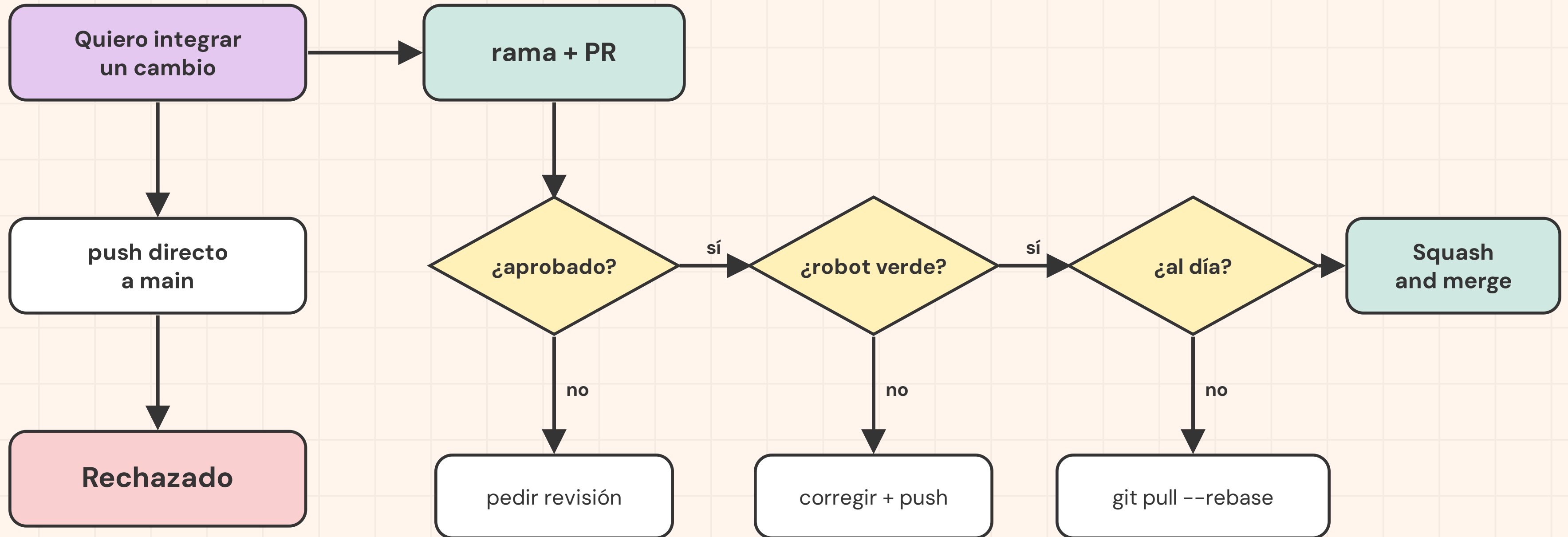
Require status checks to pass → “el robot tiene que dar el visto bueno”

Require branches to be up to date → “tu rama tiene que estar al día con main”





EL CAMINO DE UN PR

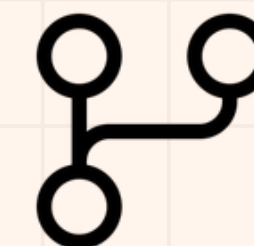


(Bloque 6)

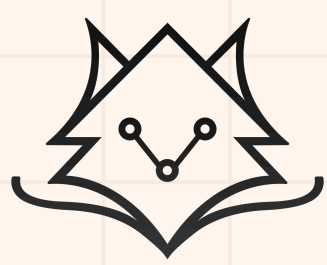
→ *Tu rama nace fresca y envejece sola. ¿Cómo la pongo al día sin ensuciar la historia?*



TU RAMA NACE **FRESCA** Y ENVEJECE **SOLA**



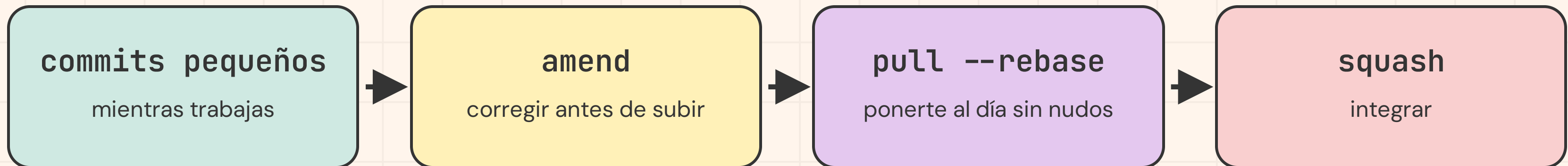
Bloque 6 · Historia limpia · 35 min



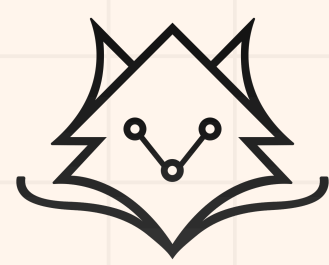
LA META

1 PR = 1 COMMIT EN MAIN

con un título claro y un **(#N)** que lleva al PR



Regla que las une: **solo se reescribe historia propia**. La de main, nunca.



SIEMPRE FAST-FORWARD

Nueva tarea, ritual completo:

issue: *Agregar mi ritual de trabajo*

```
git switch main
```

```
git pull
```

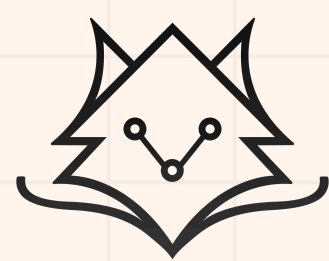
```
git switch -c feature/11-ritual-ana
```



```
$ git pull
Updating 3c4d5e6..8f9a0b1
Fast-forward
integrantes/luis.md | 12 ++++++++
```

“main solo avanza hacia adelante.”

Nadie trabaja directo en main → su `git pull` nunca crea un commit nuevo.



EL COMMIT CON UN OLVIDO

“Amend es gratis mientras el commit no haya salido de tu compu.”

 TU COMPU

X LA TENTACIÓN



```
9f1e2d3 docs: agrego mi ritual de trabajo  
4c5b6a7 ya ahora sí
```

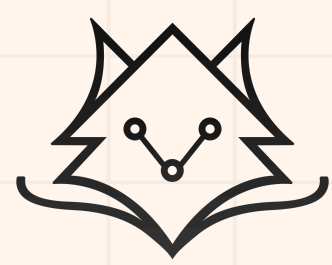
✓ MEJOR

```
git add ritual.md
```

```
git commit --amend --no-edit → “mete el cambio al último  
commit”
```

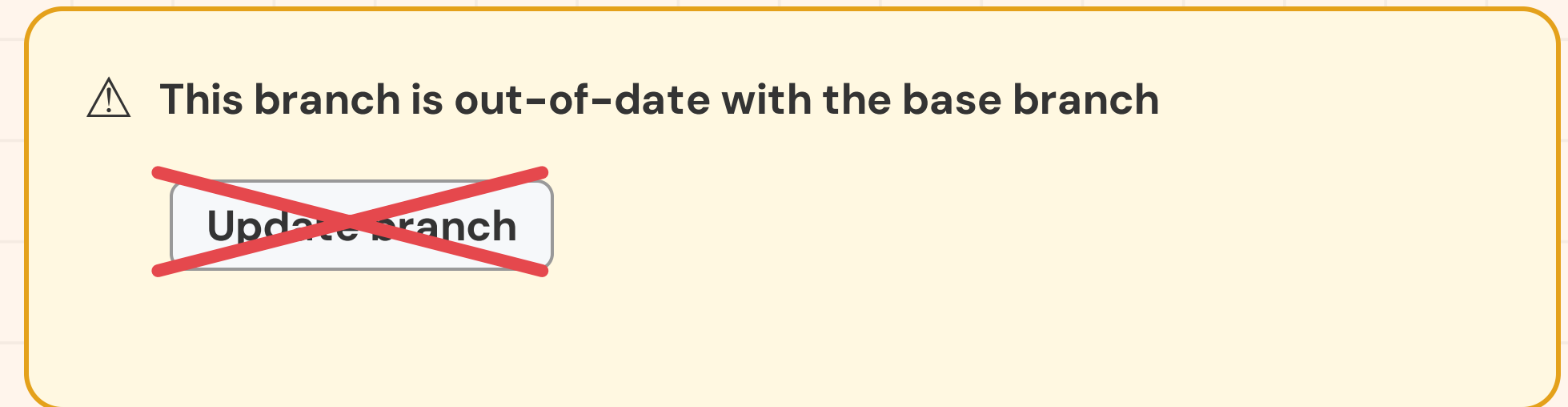
a1b2c3d → 9f8e7d6 el hash cambió: **no lo editaste, lo reemplazaste**

¿Mensaje mal? `git commit --amend -m "..."`



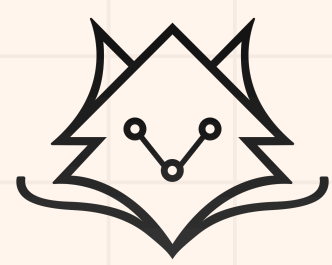
TU RAMA ENVEJECIÓ

“Lo hacemos desde la terminal, y limpio.”

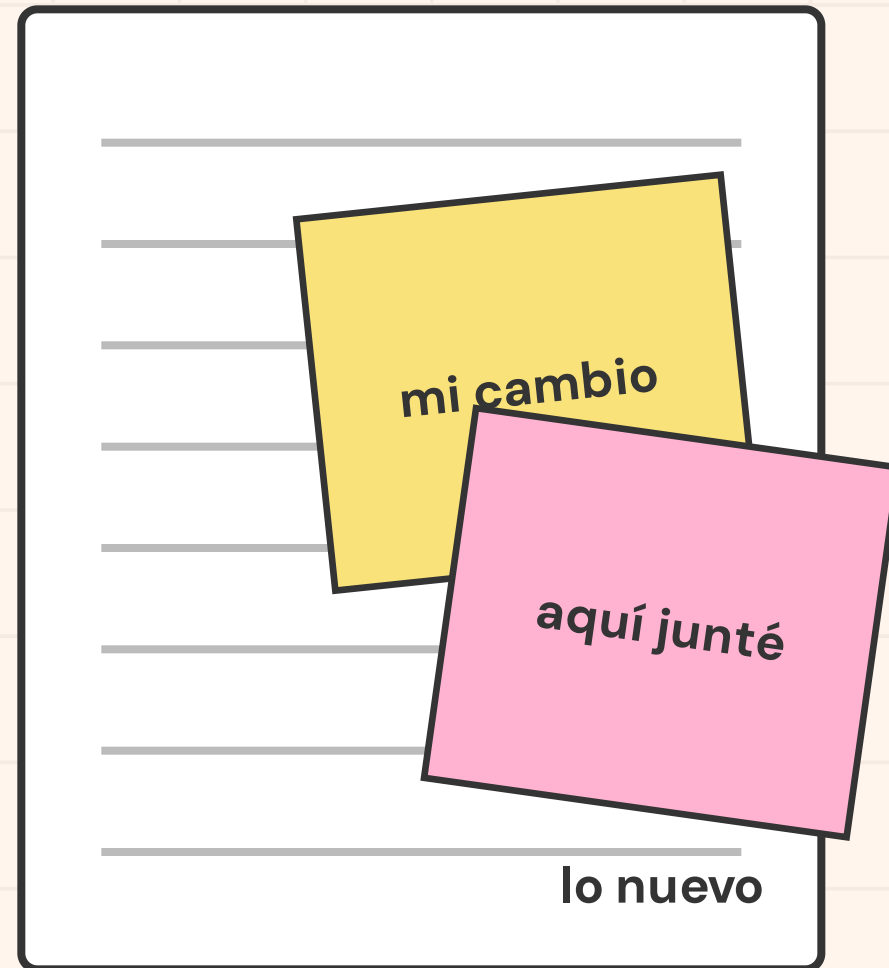


¿Por qué no ese botón?

Mete un **commit de merge** dentro de tu rama = un **nudo**.



LA ANALOGÍA DEL POST-IT



versión vieja

MERGE

otro post-it encima → **nudos**



versión NUEVA

REBASE

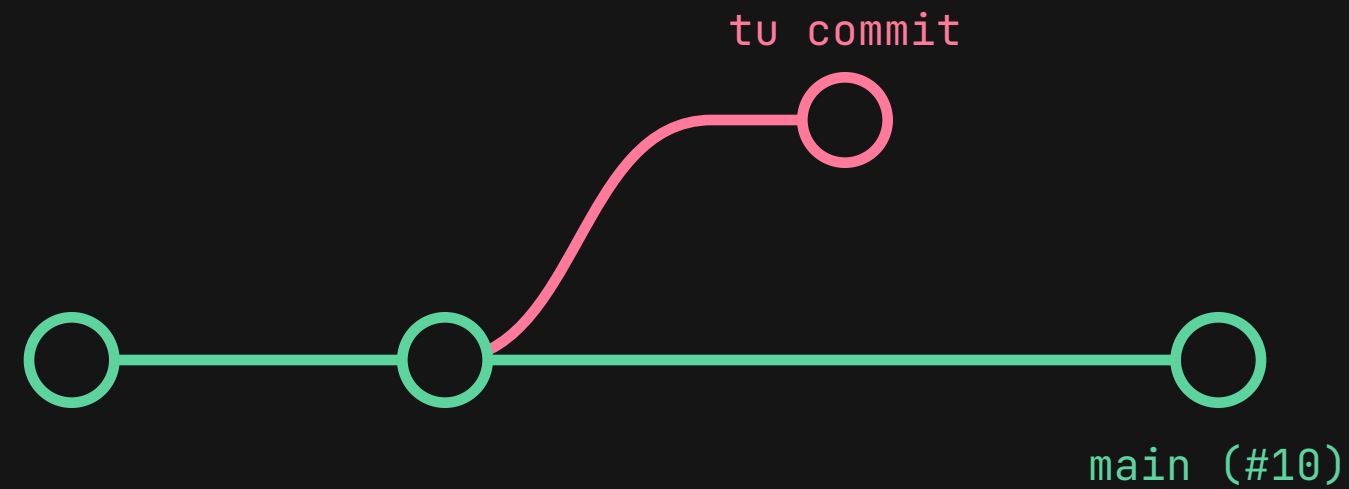
cambias la hoja de abajo → **línea recta**

despego
y re-pego



EJECUTAR EL REBASE

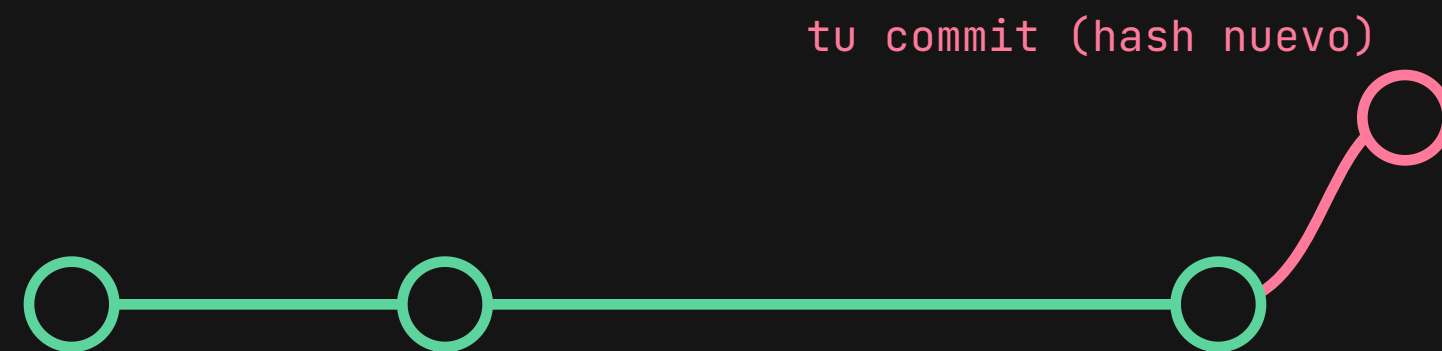
antes



```
git pull --rebase origin main → "trae main y vuelve a
pegar mis commits encima"
```

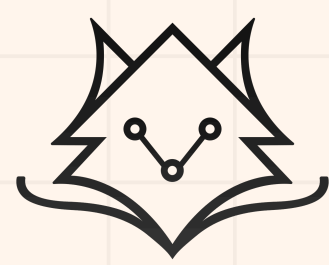
git push  rechazado... ¿por qué?
hash nuevo ≠ lo que tiene origin

después de pull --rebase



```
git push --force-with-lease → "empuja a la fuerza, pero solo si nadie más tocó mi rama"
```

En el PR: el aviso desaparece, el robot corre y el merge se desbloquea.



SQUASH AND MERGE

BLOQUE 4 · ANTES DE LAS REGLAS

docs: agrego mi ritual de trabajo (#12)

Closes #11

Confirm squash and merge

“Esto es lo que va a quedar escrito en main para siempre.”



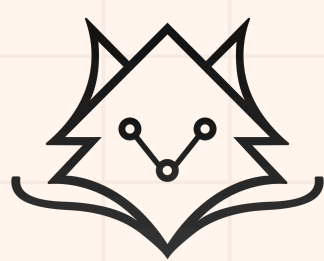
```
* c7d2e9b Merge pull request #6 from ...  
|\n| * 9b3c7d1 docs: ya ahora sí  
| * 2a8e4f0 docs: agrego perfil de Luis  
|/  
* e4f1a2c Merge pull request #4 from ...
```

AHORA



```
* 5d6e7f8 docs: agrego mi ritual de trabajo (#13)  
* 8f9a0b1 docs: agrego mi ritual de trabajo (#12)  
* 1a2b3c4 ci: agrego robot revisor (#8)
```

línea recta · un commit por PR



LA PISTA NO SE PERDIÓ

MAIN

cuenta el **qué**

EL PR

cuenta el **cómo**

EL ISSUE

cuenta el **porqué**

GitLens hover → **Open Commit on GitHub**

o `gh pr view 12 --web`

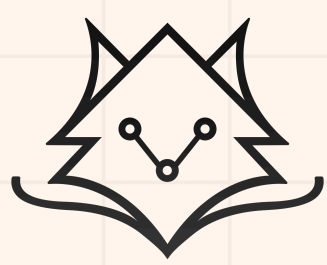
LIMPIEZA DESPUÉS DE UN SQUASH

```
git branch -d feature/11-ritual-ana
```

✗ **not fully merged** — normal: el squash creó otro hash

```
git branch -D feature/11-ritual-ana
```

→ “ya verifiqué que el PR dice **Merged**: bórrala”



REGLAS DE ORO

1

Solo reescribes **historia propia**: tu rama, antes de compartirla. La de main, nunca.

2

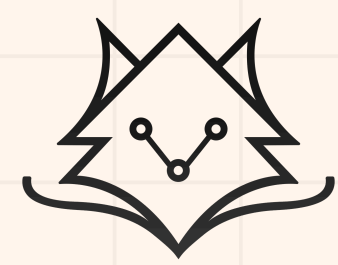
Nunca **--force** a secas.
Siempre **--force-with-lease**.

3

Una rama, un PR. Después del squash, la rama muere.

4

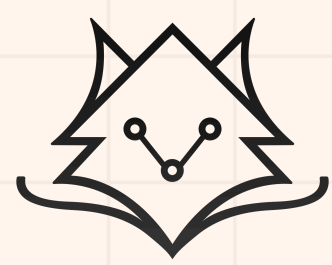
¿Algo salió raro a media operación?
git rebase --abort



MAPA DE DECISIONES

Situación	Herramienta	¿Reescribe historia?
Guardar un avance en mi rama	Commit pequeño	No
Me equivoqué en el último commit y no lo he subido	<code>git commit --amend</code>	Sí: solo mi último commit, en mi compu
main avanzó mientras yo trabajaba	<code>git pull --rebase origin main</code>	Sí: solo mi rama
Subir una rama que reescribí	<code>git push --force-with-lease</code>	Sí: solo mi rama en origin
Integrar mi trabajo a main	Squash and merge (PR)	No: agrega un solo commit
Traer main a mi compu	<code>git pull</code> en main	No: siempre fast-forward
Deshacer algo que ya está en main	<code>git revert</code> + rama + PR	No: agrega un commit que lo anula

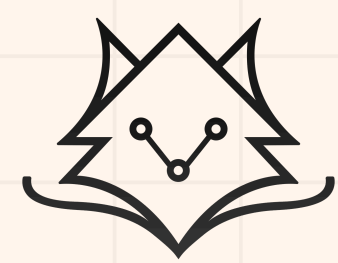
Ya que lo entendieron, volverlo costumbre: `git config --global pull.rebase true`



¿QUÉ HACE GIT CUANDO DOS PERSONAS TOCAN LA **MISMA LÍNEA?**



Bloque 7 · Conflictos en equipo · 35 min



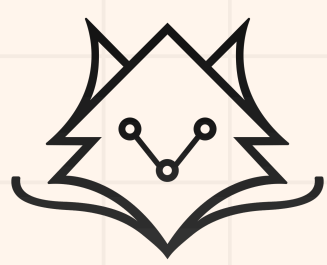
LA TAREA QUE GARANTIZA EL CHOQUE

Todos agregan **su fila** a la tabla del README, **en el mismo lugar**.

Nombre	Carrera	Usuario de GitHub
← todos aquí		

 **This branch has conflicts that must be resolved**
Conflicting files: README.md

Ritual completo: issue → rama **feature/N-fila-ana** → commit → push → PR



CONFLICT

NO ES UN ERROR

“Git no se rompió. Se detuvo a preguntarte algo que no puede decidir solo.”



Terminal

```
$ git pull --rebase origin main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
error: could not apply 4b5c6d7... docs: agrego mi fila

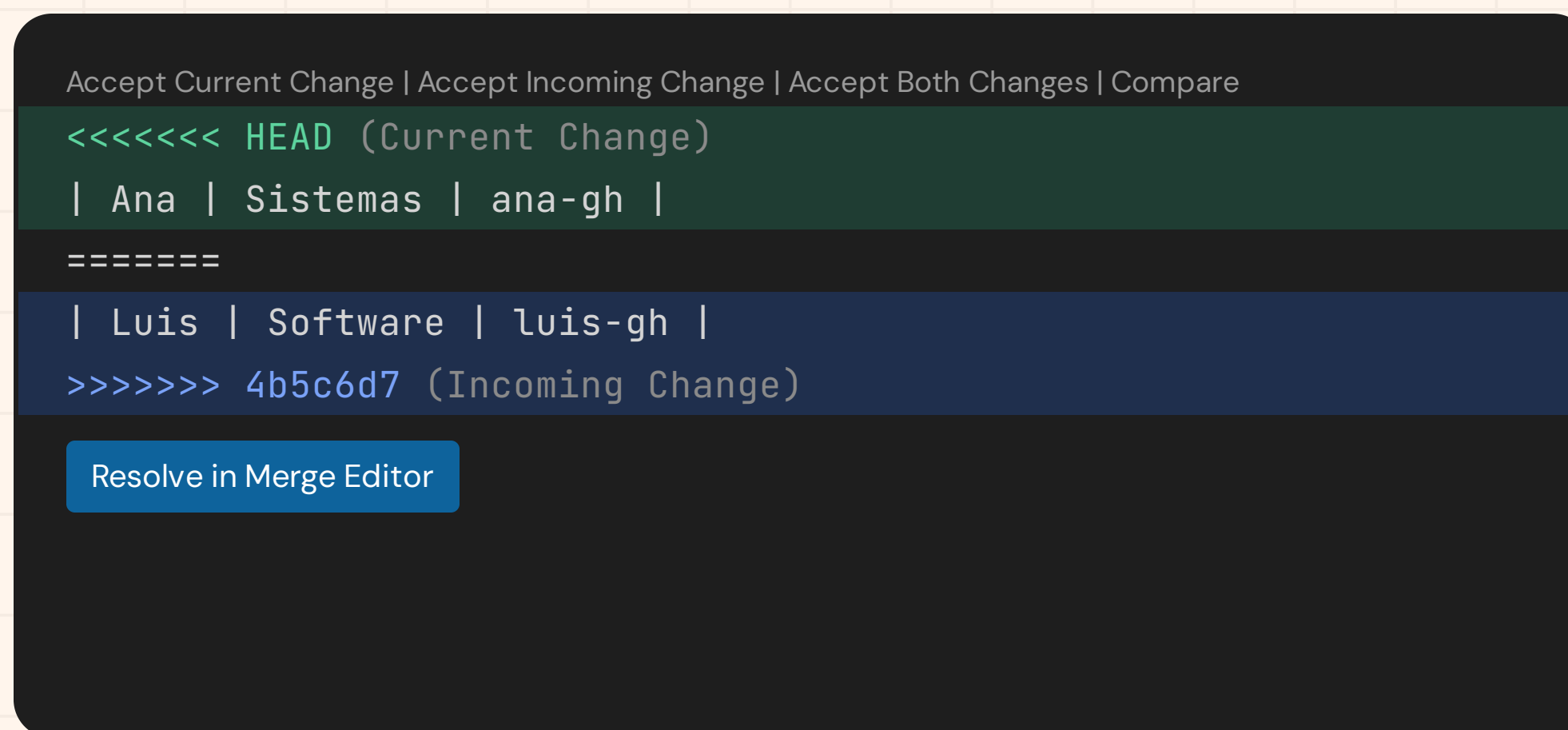
$ git status
interactive rebase in progress; onto 8f9a0b1
  (fix conflicts and then run "git rebase --continue")
  (use "git rebase --abort" to check out the original branch)
    both modified:   README.md
```

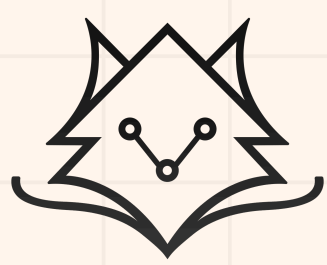


LEER EL CONFLICTO

“Dos borradores del mismo párrafo.”

<<<<<< empieza uno
===== los separa
>>>>>> termina el otro





¡OJO! EN UN REBASE SE INVIERTEN

CURRENT

= lo que **ya está** en main
(la fila del compañero)

INCOMING

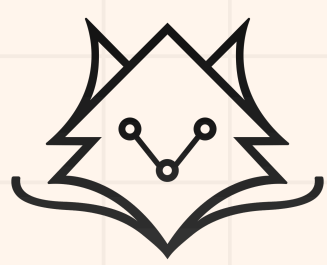
= **tu commit** que se
vuelve a pegar



Respuesta de este ejercicio:

ACCEPT BOTH

Decirlo ANTES de que abran el editor



TERMINAR EL REBASE

`git add README.md` → “ya resolví este archivo”

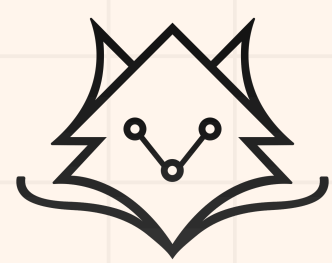
`git rebase --continue` → “sigue”

`git push --force-with-lease` → “subo mi rama reescrita”

--continue abre un editor: cerrar la pestaña de VS Code, o escribir :wq + Enter si es Vim

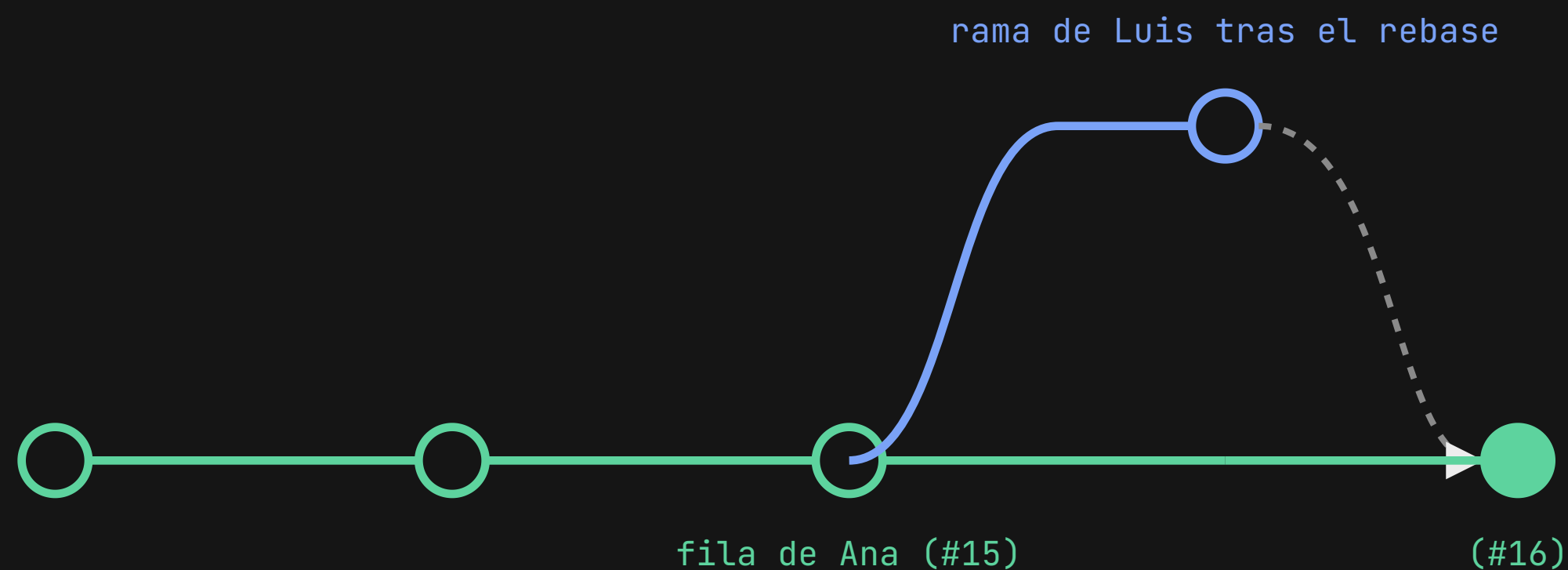
PÁNICO

`git rebase --abort`
“todo vuelve a como estaba”



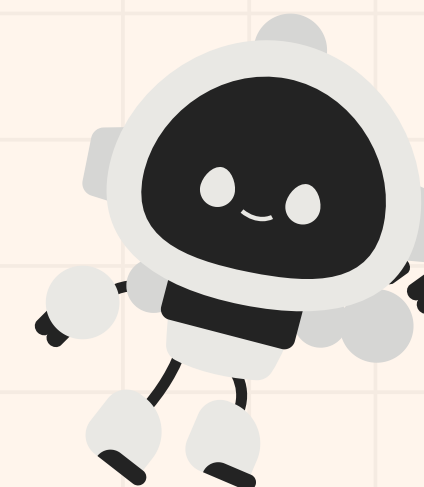
SIGUE SIENDO UNA LÍNEA RECTA

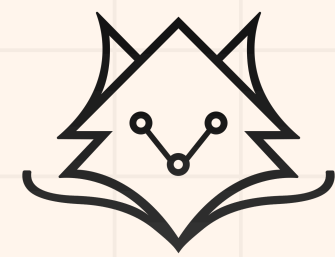
squash de Luis → un commit más en la misma línea



El conflicto se resolvió **dentro** de su commit, no con un commit extra de “arreglo el merge”.

¿Marcador olvidado? El robot se pone en **rojo**. Para eso lo contrataron.





INTEGRAR EN CADENA

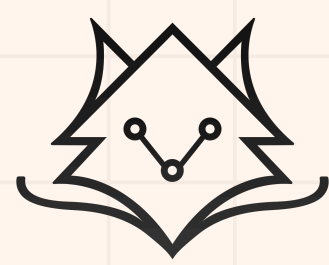


Cada autor resuelve **solo** su conflicto.

Ctrl+Shift+P → GitLens: Toggle File Blame

Luis (#2)	Nombre Carrera Usuario
Ana (#15)	Ana Sistemas ana-gh
Luis (#16)	Luis Software luis-gh
Sofi (#17)	Sofi Mecatrón. sofi-gh
Beto (#18)	Beto Actuaría beto-gh

“Cuatro personas en paralelo, con conflictos, y main sigue siendo una línea recta.”



MAÑANA, SOLOS, **¿SABEN QUÉ** HACER PRIMERO?



Bloque 8 · Ciclo completo sin red · 15 min



CICLO COMPLETO SIN RED

“Cada quien crea un issue para otra persona, y cada quien resuelve el que le asignaron. Solo con el acordeón.”

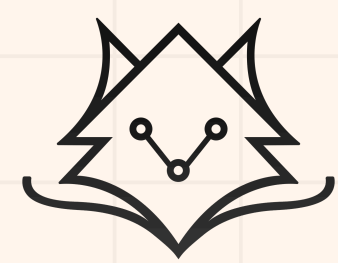
Tema: `recomendaciones/[tu-nombre].md`

un libro, video o canal

`gh issue create --assignee usuario` → “le asigno la tarea a alguien”

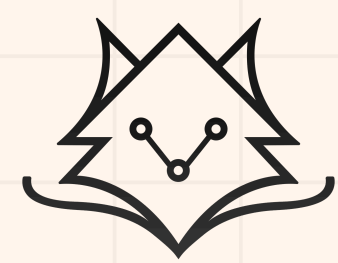
`gh issue list --assignee @me` → “¿qué me toca?”

El instructor y los asistentes solo responden preguntas: no tocan teclados



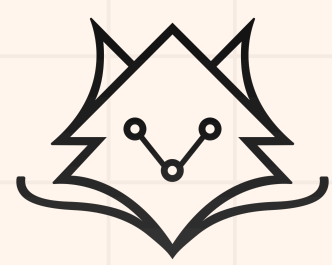
ACORDEÓN 1/2 · ANTES Y DURANTE

Momento	Qué hago	Dónde / comando
Antes	Ver qué me toca	Issues · <code>gh issue list --assignee @me</code>
	Pararme en la base	<code>git switch main</code>
	Ponerme al día (fast-forward)	<code>git pull</code>
	Abrir mi desvío	<code>git switch -c feature/[n]-[descripción]</code>
Durante	Ver qué cambió	<code>git status</code> / <code>git diff</code>
	Guardar una foto pequeña	<code>git add + git commit -m "tipo: mensaje"</code>
	Corregir mi último commit (sin subir)	<code>git commit --amend --no-edit</code>
	Ver dónde estoy	<code>git log --oneline --graph</code> / Graph
	Si main avanzó	<code>git pull --rebase origin main</code>



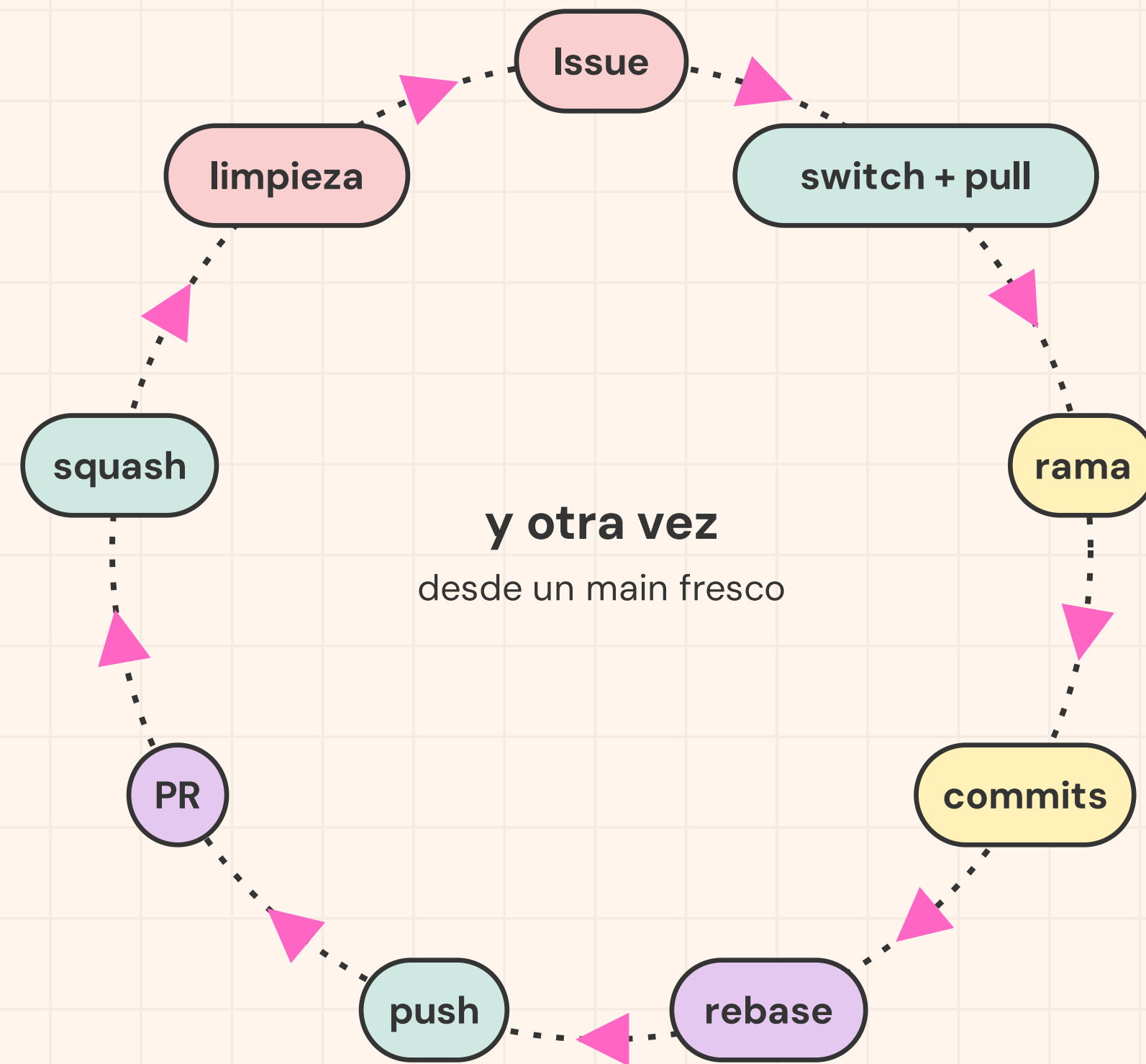
ACORDEÓN 2/2 · DESPUÉS Y PÁNICO

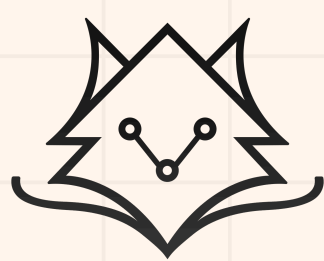
Momento	Qué hago	Dónde / comando
Después	Subir mi rama	<code>git push -u origin [rama]</code> / <code>git push</code>
	Subir tras rebase o amend	<code>git push --force-with-lease</code>
	Pedir revisión	<code>gh pr create --web</code>
	Ver el robot	Checks del PR · <code>gh pr checks</code>
	Revisar a otros	Files changed → Review changes
	Integrar	Squash and merge
	Limpiar	<code>git switch main</code> + <code>git pull</code> + <code>git branch -D [rama]</code>
Pánico	Salir de un rebase a medias	<code>git rebase --abort</code>
	Deshacer algo que ya está en main	<code>git revert [hash]</code> en rama nueva + PR
	Dejar mi main igual a origin	<code>git reset --hard origin/main</code> (tira lo local)



EL CICLO COMPLETO

"Si te pierdes, busca en qué punto del círculo estás."





LIMPIEZA Y LA PRUEBA

- ☐ `git switch main`
- ☐ `git pull`
- ☐ `git branch` → solo **main**
- ☐ `git fetch --prune`
- ☐ Issues → todo cerrado
- ☐ Pull requests → nada abierto

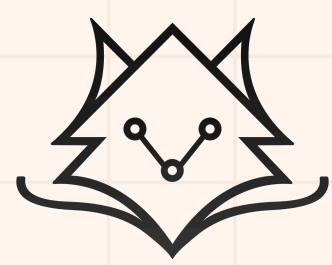
LA PRUEBA · GIT LOG --ONELINE



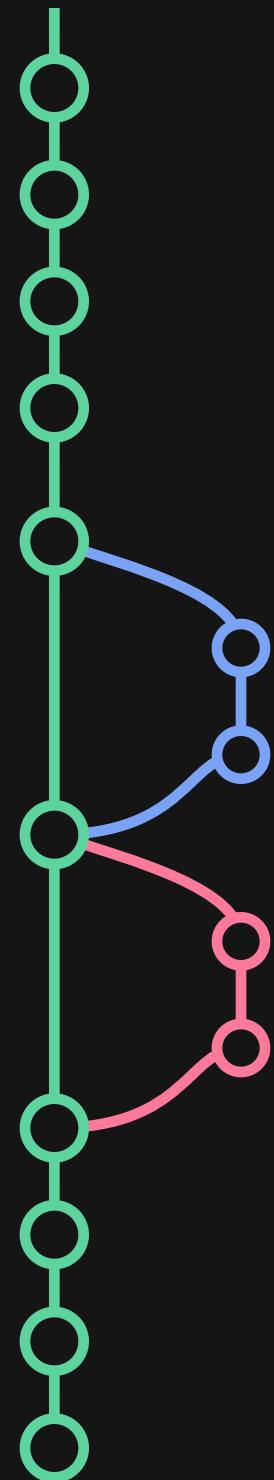
```
5d6e7f8 docs: agrego recomendación de Ana (#21)
8f9a0b1 docs: agrego fila de Beto (#18)
2c3d4e5 docs: agrego fila de Sofi (#17)
7a8b9c0 ya ahora sí
3e4f5a6 Merge branch 'main' into feature/...
1a2b3c4 ci: agrego robot revisor (#8)
```

Desde el Bloque 5: **un PR por línea.**

¿Ves un **rojo**? Algo del ritual se saltó: buen tema para la plenaria.



EL RECORRIDO



B6-7 línea recta: rebase y conflictos

B5 entra el robot, ya como squash

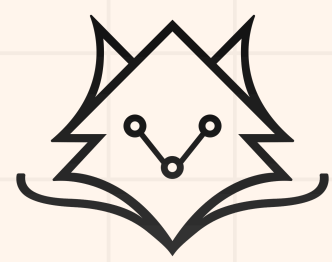
B3-4 nudos: los perfiles, antes de las reglas

B2 se publicó

B1 git init en la compu del líder

“La mitad de abajo contra la mitad de arriba.”

Proyectar la Graph real de un equipo



ES UNA ESTRATEGIA.
NO ES LA DEFINITIVA NI LA CORRECTA.
ES UNA DE MUCHAS.

issue por tarea

rama por issue

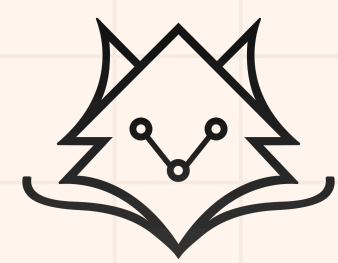
aprobación obligatoria

robot

rebase

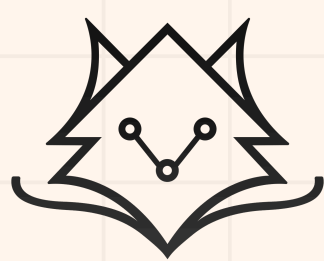
squash

Lo que elegimos hoy, porque sirve para enseñar y para un equipo chico en paralelo.



¿QUÉ CAMBIARÍA SUS REGLAS?

Factor	← menos reglas	más reglas →
Tamaño del equipo	1 a 3 personas que se sientan juntas	decenas que no se conocen
Vida del proyecto	prototipo de fin de semana	producto que vivirá años
Tiempo	hackatón de 48 h	semestre o sin fecha de fin
Burocracia aceptada	confianza, acuerdos de palabra	aprobaciones, revisores, checklists
Trazabilidad	nadie preguntará por qué	hay que explicar cada cambio
Costo de un error	se arregla en 5 minutos	afecta a usuarios reales
Frecuencia de entrega	una vez, al final	todos los días
Experiencia	todos dominan Git	hay quien está empezando



TRESEJEMPLOS

Proyecto personal

Directo en main. Nadie más necesita revisar.

Hackatón de 48 h

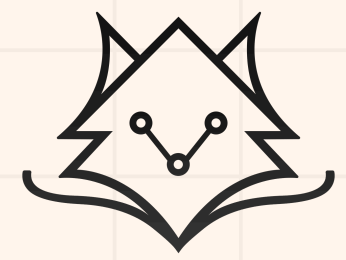
Ramas cortas, sin aprobación obligatoria. El robot quizá sobra.

Producto con usuarios

Más revisores, versiones con etiquetas, robots con pruebas de verdad.

También válido: rebase merging · merge commits · sin protección (equipo de 2 que se habla todo el día)

“Un botón les da una sola forma; la terminal les da todas. Las reglas están para servirle al equipo.”



LA TAREA

QUÉ	Proyecto propio · ≥ 3 estrategias de trabajo · ≥ 2 formas de integrar · elegir y justificar
DÓNDE	<code>docs/soporte/workflow_y_politicas_de_trabajo_colaborativo.md</code>
CÓMO	Con el flujo: issue → rama → PR revisado. El historial cuenta.
EXTRA	Documentar las reglas que decidieron no tener · ajustar la config del repo
VIDEO	5 a 10 min · con slides · todos los integrantes · explicar, no leer

ANTES DE LA SIGUIENTE SESIÓN

Sin video no se acreditan
los puntos.

Referencia: `material/tarea.md`



RECURSOS Y DESPEDIDA

`gh help · gh [comando] --help`

Docs de GitHub: flujos, rulesets, métodos de merge

Pro Git en español: ramas y remotos

Oh My Git! para practicar rebase jugando

practica-terminal es su repo para equivocarse sin consecuencias.

***“Lo que aprendieron hoy no es
Git, es trabajo en equipo.”***

¿Preguntas? También como **issue** en el repo del curso.



THANK YOU



git



@WEWOLF_DEV



GitKraken